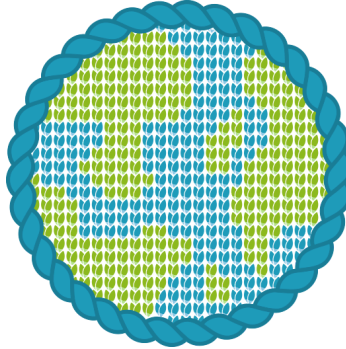


World's Tapestry, A Community Forum Website



Group 12: Chase Brown, Sean Kehoe, Jennifer Orlando, Mohammed Qudaih, and Rory Strachan

Oakland University

CSI 4999: Senior Capstone Project

Dr. Hadeel Jawad

April 5th, 2026

Silent Video Link: <https://www.loom.com/share/39468c886df14c7c8a50b3440c3c42e9>

Voiced Link: <https://www.loom.com/share/2101ba031126486183a453cf94f2bb68>

Contents

Contents.....	2
Project Description.....	6
Targeted Audience.....	6
Similar Product.....	6
Discord.....	6
Reddit and Stack Exchange.....	6
Facebook.....	6
Source of Revenue.....	6
User Types.....	7
User.....	7
Forum Member.....	7
Forum Manager.....	7
subclass: Forum Owner, Forum Admin.....	7
Software Architecture.....	7
Subsystems.....	7
Forum Admin Tools.....	7
Feature: Forum Creation and Settings.....	8
Feature: Tags.....	9
Feature: Member Removing / Member Role Change.....	9
Feature: Adding Categories.....	10
Forum interaction.....	10
Feature: Joining.....	10
Feature: Posting.....	10
Feature: Liking and Disliking.....	11
Feature: Post Editing.....	11
Map.....	11
Feature: Pin Placement.....	11
Feature: Road status updates.....	12
Feature: Legend.....	12
Feature: Reviews.....	12
User Profiles and Direct Messaging.....	12
Feature: User Profiles.....	13
Feature: User settings.....	13
Feature: Friending Users.....	13
Feature: Direct Messaging.....	13

Website Interaction and Moderation.....	13
Feature: Reporting Forums, Threads, and Posts.....	13
Feature: Reporting Bugs.....	14
Feature: Content Searching.....	14
Feature: User Blocking.....	14
Technology.....	14
IDE.....	14
Communication and Collaboration.....	14
Database.....	14
Development Language.....	14
AWS Relational Database Service.....	14
Cloudinary.....	15
Use Case Diagrams.....	15
Use Case vs SQL Query.....	20
Implemented Use Cases + SQL Queries.....	20
System Sequence Diagrams.....	32
Subsystem: Map - Chase Brown.....	32
Subsystem: Forum Interaction - Sean Kehoe.....	33
Subsystem: Website Interaction and Moderation - Jennifer Orlando.....	34
Subsystem: User Profiles and Direct Messaging - Mohammed Qudaih.....	35
Subsystem: Forums Admin Tools - Rory Strachan.....	36
Activity Diagram.....	37
User Interface Design.....	38
Wireframes.....	38
Mockups.....	39
Responsive Design.....	41
HCI Rules Discussion.....	41
Keep Users in Control.....	41
Permit Easy Reversal of Actions.....	41
Strive for Consistency.....	42
Final Screenshots and Storyboard.....	43
Database Design.....	53
All Tables.....	53
Table: Forums (Main table).....	53
Table: Categories.....	53
Table: BugReport.....	54
Table: Locations.....	54
Table: Maps.....	54

Table: MemberList.....	55
Table: Messages.....	55
Table: PlaceReviews.....	56
Table: PostRatings.....	56
Table: Posts.....	56
Table: Reports.....	57
Table: Threads.....	57
Table: UserPins.....	57
Table: UserRelationships.....	58
Table: Users.....	58
Entity Relationship Diagram (ERD).....	59
Sample Code.....	59
Chase Brown.....	59
Data loading functions:.....	59
Pin related functions.....	61
Map interaction functions.....	65
useEffect hooks.....	66
Sean Kehoe.....	67
Post Editing.....	67
Post Creation.....	70
Rory Strachan.....	73
Changing User Roles Within A Forum.....	73
Editing Forum Tags.....	81
Jennifer Orlando.....	84
About Us Page.....	84
Report Feature.....	86
Mohammed Qudaih.....	88
User Profile Viewing.....	88
CODE REVIEW 2.....	90
ADDIE Model Discussion.....	90
Analysis.....	90
Design.....	90
Development.....	91
Implementation.....	92
Evaluation.....	92
Future Updates.....	92
Map.....	92
User Personalization.....	92

Safety Features.....	93
Events.....	93
Appendices.....	94
Appendix A: System Vision Document.....	94
Problem Description.....	94
System Capabilities.....	94
Business Benefits.....	95
Appendix B: Work Breakdown Structure.....	95
References.....	97

Project Description

Targeted Audience

Users aged 18-40 who wish to engage with others online.

Similar Product

Most of the similar systems do not have a map feature to find events and communities. And those who claim to have a map feature only use the user's location data and do not display a map. Our app will have an interactive map which our users can use as a visual gateway to the community forum.

Discord

This app utilizes communities with various chat chains. This app's focus on user created communities is a feature we admire and are aiming to make something similar.

Reddit and Stack Exchange

These apps' main focus is the forum. Our app will have other features such as events.

Facebook

This app is the most similar to what we are creating. Our app however will have a different layout and less emphasis on user profiles.

Source of Revenue

Our project will have multiple sources of revenue. One source of revenue will be targeted ads towards our users, another source of revenue will be donations from our users, and the final source of revenue will be post bumping from forum administrators.

User Types

User

Users are a generic superclass for all profiles/users of the site regardless of any other status. (I.e. a fresh made account lacking membership to any forum, admins, members, etc.) This generic account will only be allowed full post privileges on any forums listed as publicly available and only able to read posts on those that allow only members to post and are prohibited from any that are listed as strictly private.

Forum Member

Members make up the majority of users of our site and are granted post, commenting, and like/dislike privileges to any forums which they belong to.

Forum Manager

subclass: Forum Owner, Forum Admin

Forum managers are split into two types: Administrators and Owners. Administrators are those appointed by the original creator of the forum to a position of authority in which they will be granted extra permissions by the owner to moderate thus enabling them to remove posts, issue bans and suspensions to members/users, and create special status posts such as those pertaining to security issues or events. Owners meanwhile have all the same powers as administrators in addition to the ability to fully remove the forum should they deem it necessary.

Software Architecture

Subsystems

Forum Admin Tools

As described previously, the users of the web application will be members, forum admins, and super admins. Any member can become a forum admin if they choose to make a

forum. The admin tool's subsystem describes the set of functions that allow users to create and manage forums.

Feature: Forum Creation and Settings

For the proposed website to fulfill its purpose, there must be an interface that allows for forum creation. Nearly any person who uses the website should be allowed to create a forum, there will be some level of restriction for users that have been known to cause problems and have been reported often. The users will be able to go to a specific page to fill out a form that will assist in tailoring the forum to the users wants.

One primary setting will be privacy. Forums will be able to be one of the following types in Table 1.

Type	Search Visibility	Post Reading Restrictions	Posting Restrictions
Public	Viewable in Forum Search	Anyone	Anyone
Read only	Viewable in Forum Search	Anyone	Members Only
Private	Viewable in Forum Search	Members Only	Members Only
Completely Private	Hidden from Forum Search	Members Only	Members Only

Table 1: Forum privacy setting types

The listed privacy settings involve how the forum is viewed on the website. There will be another set of privacy settings pertaining to how members join the forum. The join permissions are listed in Table 2.

Type	Who Can Join	Notes
Open	Anyone	
Restricted Open	Users Who Meet Forum Requirements	The user will only be allowed to join if their account meets certain requirement(s). The requirement could be user age or account creation date (to only allow accounts that have existed for more than x amount of days).
Invite Only	Invited Users	The user must be invited into the forum by an admin.

Table 2: Forum join permissions

The forum will also provide a few extra settings. There will be options for if the forum will allow liking and or disliking posts, if the forum wants to utilize the map or news features, and creation of forum specific user types.

Ideally, each option will be allowed to be altered after the forum creation within a forum setting widget.

Feature: Tags

Upon forum creation the user will be prompted to set tags for the forum. These tags can be used to let users know what the “purpose” of a forum is and what the forum caters to.

Once set, new tabs can be added and old tags can be deleted through the admin panel.

Feature: Member Removing / Member Role Change

Forum owners and admins will have access to the admin panel. Through the member settings within the admin panel, these admins will be able to see all members that have joined the forum.

Admins will be allowed to remove members from the forum. The owner can remove admins and users of the forum, admins can only remove users from the forum.

The member settings page also allows admins to change the roles of users. Owners and admins will be allowed to promote users to an admin role.

Feature: Adding Categories

Threads in forums can be organized into different categories. Forum administrators and forum owners will determine what these categories are.

Category addition is a simple process where forum administrators and owners can enter data into a forum to add a new category row in the database. Users will be able to add posts to the category as soon as it is created.

Forum interaction

Forum interaction forms the backbone of the site's interactivity as it allows users of all kinds to post, comment, and interact with one another. Posts made on the forum will form a thread of comments and replies that allow other users to respond and create further dialog. Additionally, posting privileges will be assigned based on the privacy settings created during the initial forum setup wherein they could be restricted to members of the forum or available to all users.

Feature: Joining

Users will be allowed to join forums. For private forums, joining is the only way the user would be able to have access to a forum. For public forums joining does not make a huge difference, but joining a forum will make it more visible on a user's home page. Joining a forum may also signal to other forum members that a user would like to be involved in their community.

Feature: Posting

In order for a forum to be a viable place to exchange ideas freely a means of public communication must be established. Therefore, depending on the privacy settings of the forum

users will be able to create posts about subjects pertinent to the forum (i.e. replies, new topics about the neighborhood, etc.).

Feature: Liking and Disliking

Forums require a simple and effective means for their members to engage outside of the spoken word to provide quick feedback to others without having to write a response. In order to facilitate that, forum posts will implement a simple like and dislike system where users will be able to give a simple like or dislike to show approval or disapproval of a post.

Feature: Post Editing

As users post on the site they will often make errors such as spelling mistakes or wish to add clarification to their original points. To that end it is important that they be able to rectify their mistakes in a timely manner so an editing feature will be implemented that allows them to change the content of their posts.

Map

Our website features a map that acts as a visual gateway into each forum. Powered by Google Maps API, and MDOT API, we provide users with a simple and intuitive way to explore local businesses, events, roadways, and user designated areas within the community. Rather than searching through multiple posts or pages, users can rely on the map to quickly understand what is happening within their community and navigate the area more efficiently. This map feature strengthens local engagement, improves accessibility, and provides users with features such as pin placement, reviews, real-time road status updates, and a legend.

Feature: Pin Placement

The pin placement feature within our map will allow users to mark specific locations on the map that are relevant to the forum they are a member of. Including meet up locations, points

of interest for specific groups (trails for hikers, coffee shops, dog parks, etc.). Each pin includes basic information such as a title, description, reviews of the area, and time stamps. This feature promotes community engagement and is an easy way to keep the map current.

Feature: Road status updates

Our live road status updates feature allows users to see where construction zones, closed roads, and accidents are on their map so they can easily navigate around the community without running into inconveniences. Our road status updates feature is powered by MDOT's API which provides us with live road status data every few minutes. This feature promotes safety by helping our users be aware of the road conditions before ever needing to step foot outside.

Feature: Legend

Our legend serves as a guide to help users interpret the information that is being displayed on the map by explaining the different icons and symbols used for pins and the various road statuses. The legend helps maintain consistency across the interface and ensures that map data is understandable for everyone.

Feature: Reviews

The maps review feature allows users to share feedback and experiences tied to specific map locations specified by other users. Users have the ability to rate a point of interest from one to five stars and type out their thoughts on each review. This feature also encourages community engagement by combining ratings and written comments while promoting accountability for the businesses and locations nearby.

User Profiles and Direct Messaging

The platform exists to support communication between groups of users. Group interaction is the primary goal, but effort will still be placed into user to user interaction.

Feature: User Profiles

When creating an account users will be instructed to list their email, password, and desired username. The profile displays the user's user name and a message field if the user is viewing another user's page. If the user is viewing their own page their profile will show a list of settings.

Feature: User settings

The user profile page features numerous settings. The user is able to change their username, email, and profile picture.

Feature: Friending Users

The platform allows users to friend other users if they so choose to. If two people mutually follow each other, they will become friends.

Feature: Direct Messaging

Unlike forum interaction which is between multiple users, direct messaging will only allow for communication between two users. There will be no threads, the system will be very simplistic.

Website Interaction and Moderation

Moderation is an integral part of any forum. There will be various tools in place to report and manage bugs, users, posts.

Feature: Reporting Forums, Threads, and Posts

Users are able to report forums, threads, and posts. The report will take in the reported content type, the reported content id, and message from the reporter.

Feature: Reporting Bugs

Users are able to report bugs they encounter. This data is sent to a bug table in the database for the website managers to fix whenever they can.

Feature: Content Searching

Users can enter text into the search bar to view all forums, categories, threads, and posts that contain the text in their titles, contents, or descriptions. At a later point filtering will be implemented to allow the user to narrow their search.

Feature: User Blocking

When viewing a profile a user is able to block another user. If a user is blocked, they cannot follow or message the user that blocked them.

Technology***IDE***

Visual Studio Code is the IDE that we used to write and deploy our code.

Communication and Collaboration

Discord is used as our primary place of communication. Github is used for hosting our code and allowing each group member to access the group individually.

Database

MySQL is the program that is used to house our database.

Development Language

The website was developed using ReactJS. HTML and CSS were used along with react tools.

AWS Relational Database Service

We are hosting our database using the AWS Relational Database Service, taking advantage of this AWS Platform as a Service allows all members of the group access the database to modify

tables for their subsystem and create relationships between tables. Hosting our database with AWS also gives us the ability to avoid SQL version mismatching, gives us high availability to avoid relying on one group members machine for testing and allows us to backup our data in the cloud with ease providing further data retention.

Cloudinary

We Cloudinary to host images for our application. We only store the part of the URL unique to each image within the database. Storing the link rather than the image itself is considered best practice in the industry and provides better performance and scalability for future growth of our application.

Use Case Diagrams

Use cases for functions are going to vary depending on the forum permissions. A forum that is public and open may allow anyone to post and comment wherever they would like. More controlled forums may only allow certain users to post in certain categories.

Typically, a forum member will be allowed to post and read from a forum that they are a member from. An administrator will be given more functionalities and permissions within the forum. The forum owner will be given absolute permissions for the forum that they own. The user with access to nearly all functionality would be the super admins, who act as both IT support and overall site moderation.

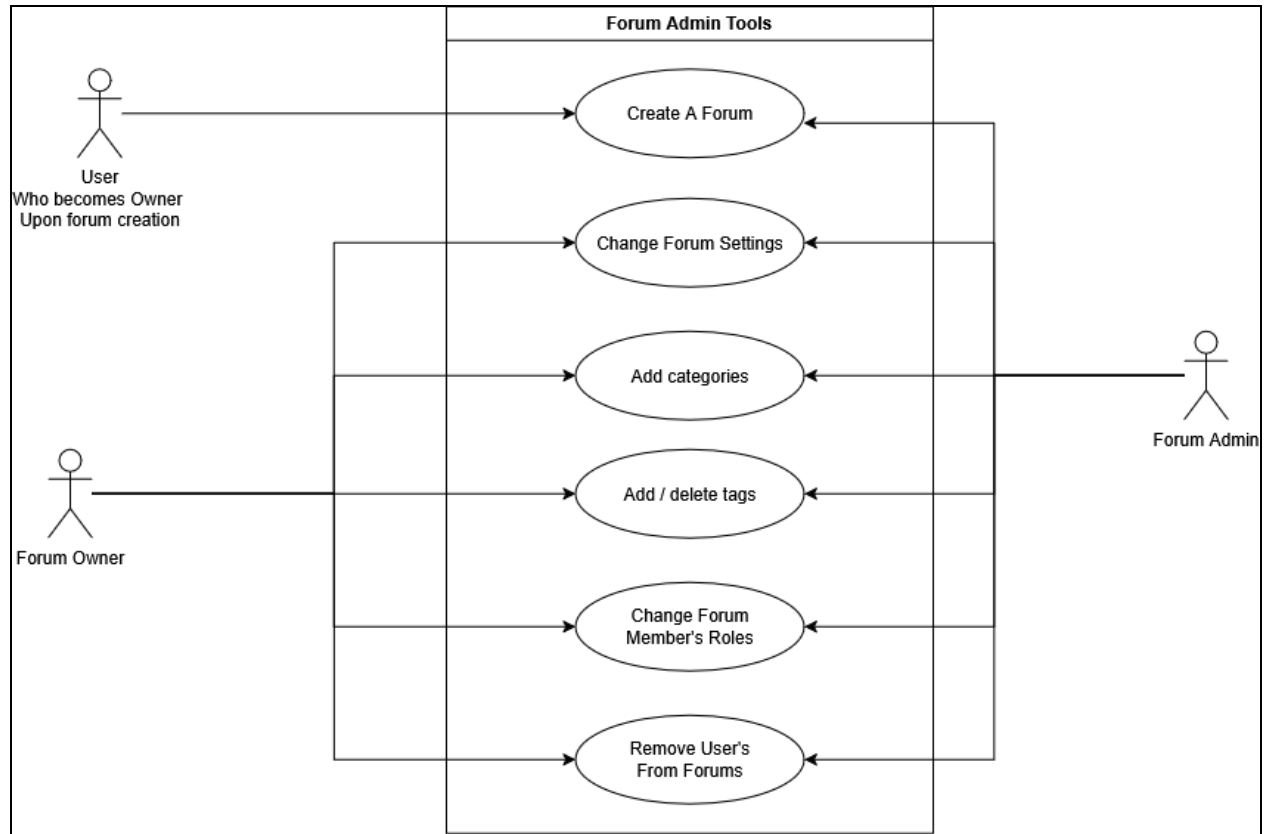


Figure 1: Forum Admin Tools Use Case Diagram

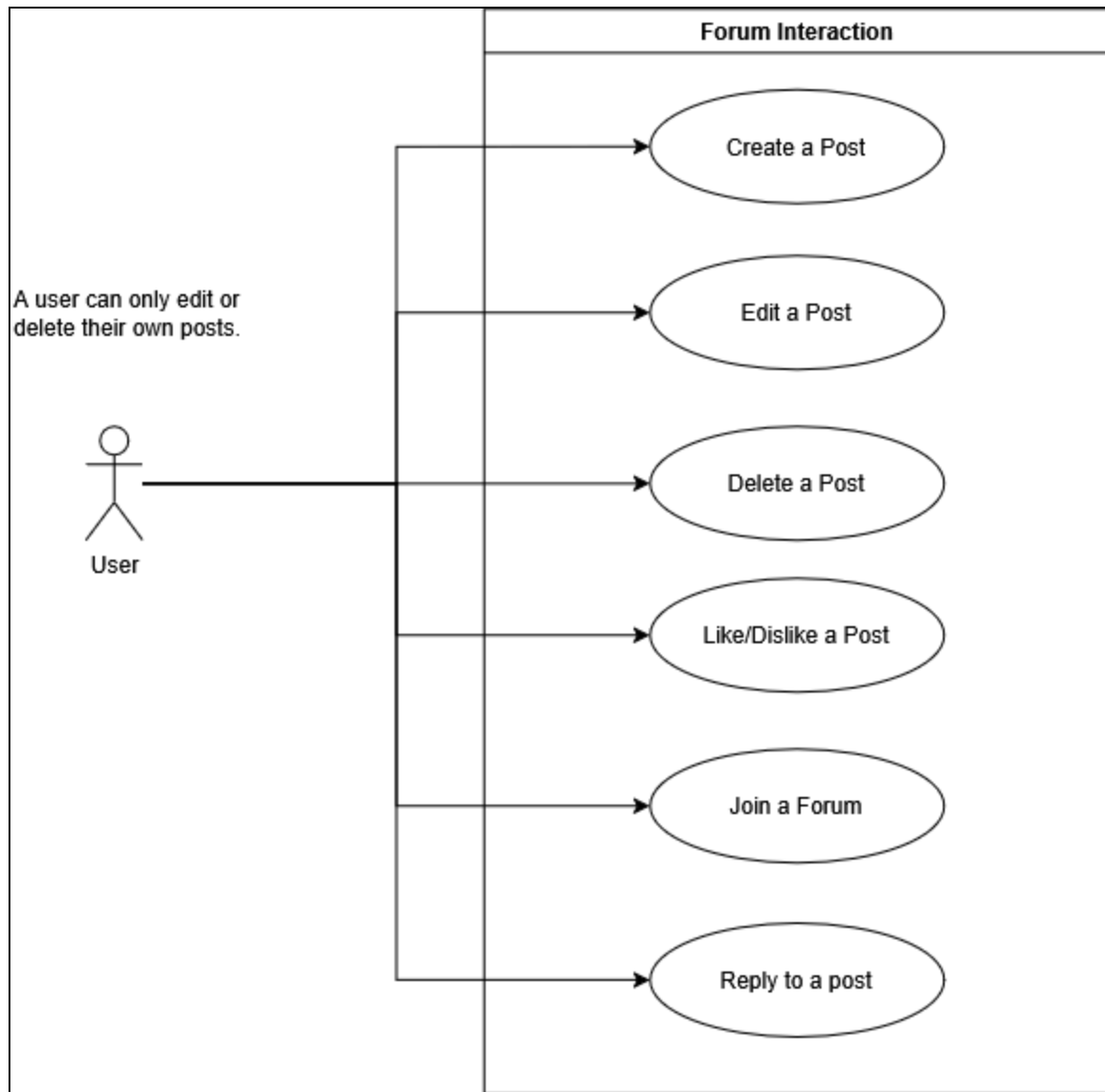


Figure 2: Forum Interaction Use Case Diagram

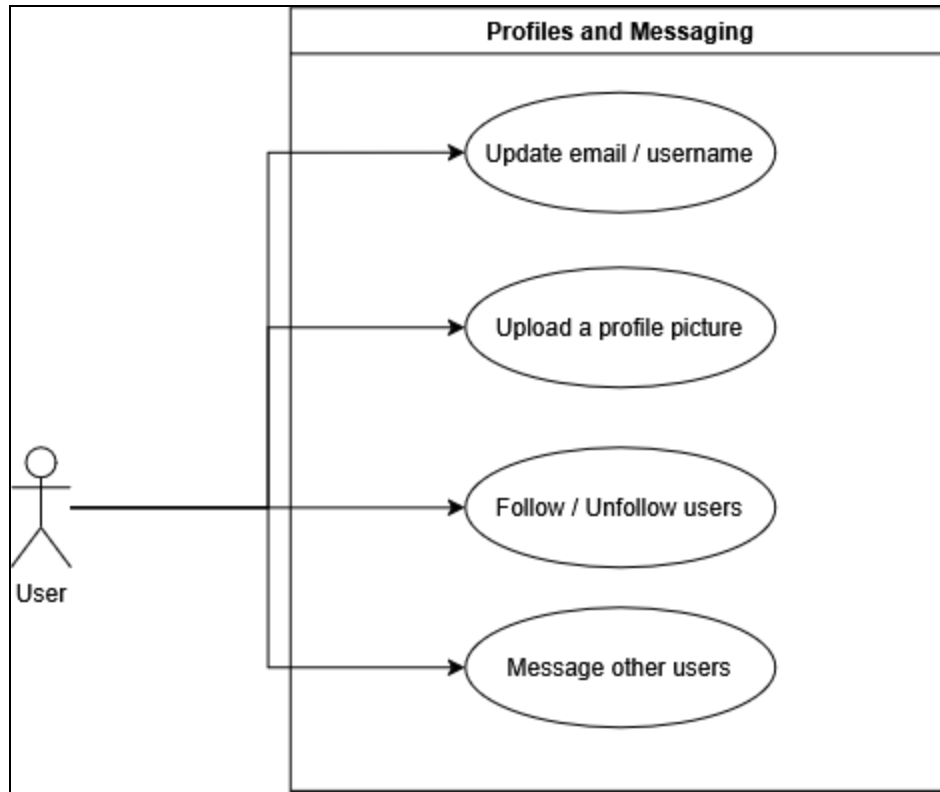


Figure 3: Profile Use Case Diagram

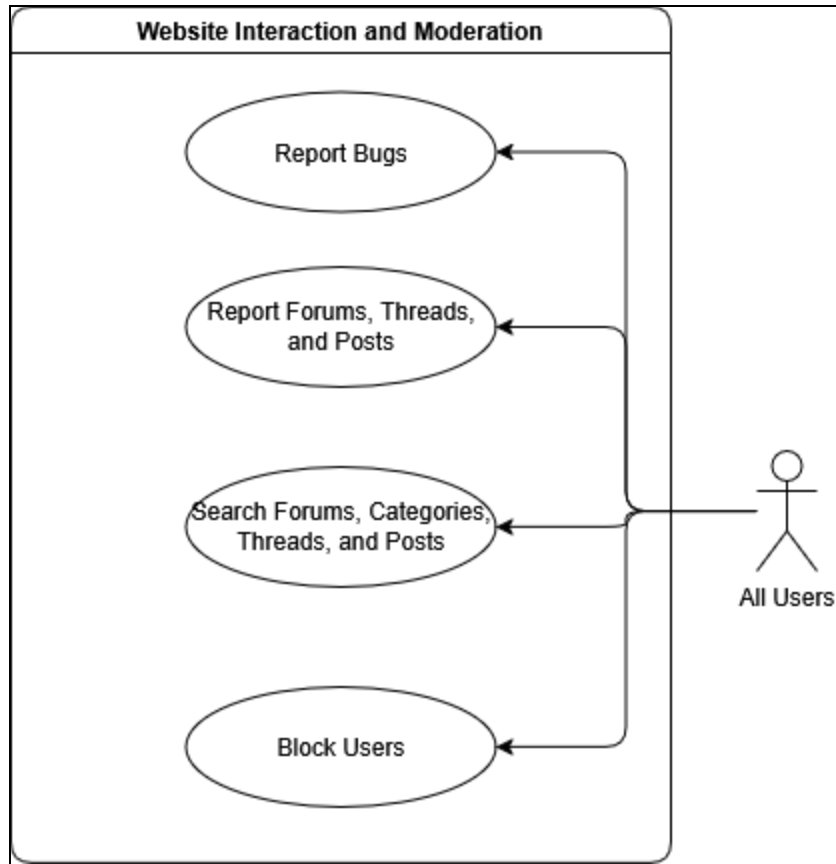


Figure 4: Website Interaction and Moderation Use Case Diagram

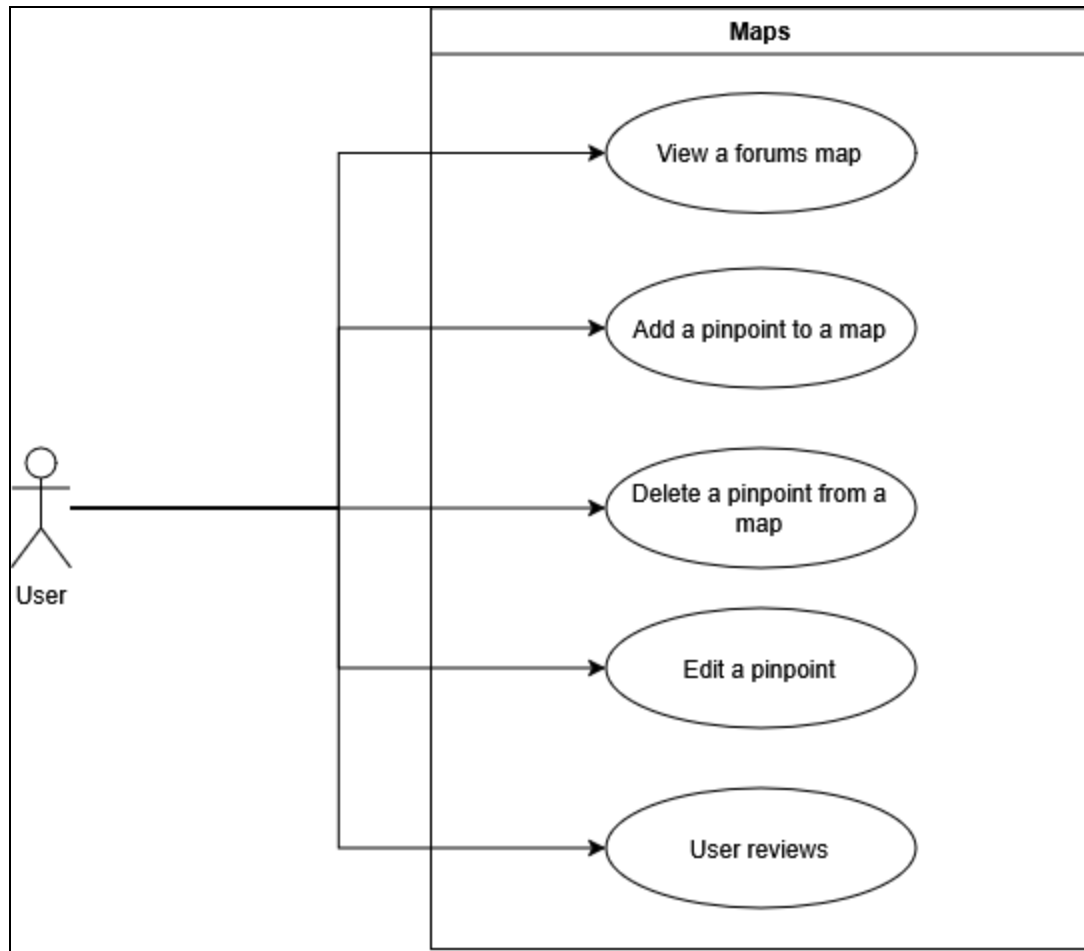


Figure 5: Maps Use Case Diagram

Use Case vs SQL Query

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
1	Forum Creation	INSERT INTO Forums (ForumName, SearchVisibility, JoinPermissions, AllowMaps, OwnerID, Tags) VALUES (?, ?, ?, ?, ?, ?) After Insert	Forums (MemberList and Maps after insert through trigger)	Frontend: LandingPage.jsx Backend: Server.js	Users can create a forum that they will become the owner of. A row will automatically be created in MemberList, since the user created a forum, they automatically become a member of that forum.

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		trigger in MYSQL: INSERT INTO MemberList (ForumID, UserID, UserRole) VALUES (NEW.ForumID, NEW.OwnerID, "Owner"); IF (NEW.AllowMaps = 1) THEN INSERT INTO Maps (ForumID) values (NEW.ForumID); END IF;			They will have the "owner" role in their forum. If the forum allows maps, a row will also be created in the map table corresponding to the forum.
2	Category Creation	INSERT INTO Categories (CategoryName, Description, Pinned, ForumID) VALUES (?, ?, ?, ?);	Categories	Frontend: ForumAdminPage.jsx Backend: Server.js	Forum admins and forum owners can create new categories for their forum.
3	Remove Users From Forums	DELETE FROM MemberList WHERE ForumID = ? AND UserID = ?; UPDATE Forums SET MemberCount = MemberCount - 1 WHERE	Memberlist, Forums	Frontend: ForumAdminPage.jsx Backend: Server.js	Forum admins and forum owners can remove members that have joined their forum. Admins cannot remove other admins or the forum owner. The forum owner can remove admins. Forums are also updated

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		ForumID = ?;			to keep track of the member count. This is done from the initial query instead of a SQL trigger.
4	Change User Roles In Forums	UPDATE MemberList SET UserRole = ? Where UserID = ? AND ForumID = ?;	Memberlist	Frontend: ForumAdminPage.jsx Backend: Server.js	Forum admins and forum owners can change the roles of members that have joined their forum. Admins cannot change the roles of other admins or the forum owner. The forum owner can change admin roles.
7	Forum Joining	INSERT INTO MemberList (UserID, ForumID, UserRole) VALUES (?, ?, ?); UPDATE Forums SET MemberCount = MemberCount + 1 WHERE ForumID = ?;	MemberList, Forums	Frontend: ForumLandingPage.jsx Backend: Server.js	Users can join a forum. This will place the forum on the user's landing page.
8	Forum Leaving	DELETE FROM MemberList WHERE ForumID = ? AND UserID = ?; UPDATE Forums SET MemberCount = MemberCount - 1 WHERE ForumID = ?;	MemberList, Forums	Frontend: ForumLandingPage.jsx Backend: Server.js	Users can leave a forum. This will remove the forum on the user's landing page. The user will lose the role they have in a forum.

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
9	Post liking and disliking	UPDATE Posts SET likes = ?, dislikes = ? WHERE PostID = ?;	Posts	Frontend: ThreadPost.jsx Backend: Server.js	Users can like and dislike posts. Users will only be allowed to like or dislike a post once.
10	Post creation	INSERT INTO Posts (UserID, ThreadID, Content) VALUES (?, ?, ?); MYSQL Triggers: UPDATE Threads SET NumberOfPosts = NumberOfPosts + 1, MostRecentPost = NOW() WHERE ThreadID = NEW.ThreadID; UPDATE Threads JOIN Categories ON Threads.Category ID = Categories.Categ oryID SET Categories.Numb erOfPosts = Categories.Numb erOfPosts + 1, Categories.Most RecentActivity = NOW()	Posts, Threads, Categories	Frontend: ThreadPost.jsx Backend: Server.js	Users can add a post to a thread and contribute to the threads conversation. My SQL triggers update: 1) The number of posts within the thread row that corresponds to the thread the post is in. 2) The most recent post time within the thread row that corresponds to the thread the post is in.

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		WHERE ThreadID = NEW.ThreadID;			
12	Post editing	UPDATE Posts SET Context = ? WHERE PostID = ?;	Posts	Frontend: ThreadPost.jsx Backend: Server.js	Users can edit their posts to have different content. The post will have an indication that it has been edited. The query is done through a patch request.
13	Post deleting	UPDATE Posts SET Deleted = 1 WHERE PostID = ?;	Posts	Frontend: ThreadPost.jsx Backend: Server.js	Users can delete their posts. This post will still be in the database. Where the post originally was within a thread will be replaced with "This post was deleted by the user". The query is done through a patch request.
14	Profile Viewing	Select * FROM Users WHERE UserID = ?;	Users	Frontend: ProfilePage.jsx Backend: Server.js	Users can view their profile and other users profile.
15	Profile Settings Update	UPDATE Users SET UserName = ? Email = ? WHERE UserId = ?;	Users	Frontend: ProfileUpdateFo rm.jsx Backend: Server.js	Users can change their profile privacy settings. This query is done through a patch request.
16	View map	SELECT * FROM Maps WHERE ForumID = ?;	Map, UserPins, Forums	Frontend: ForumMapPage. jsx	Users can view forum maps that will be populated by user pins and road statuses.

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
				Backend: Server.js	
17	Create pin	INSERT INTO Userpins (UserID, Visibility, Longitude, Latitude, Title, Description) VALUES (?, ?, ?, ?, ?, ?);	UserPins	Frontend: ForumMapPage. jsx Backend: Server.js	Users can place pinpoints onto the forum's map.
18	Edit pin	UPDATE UserPins SET Title = ?, Description = ?, Visibility = ? WHERE PinID = ? AND UserID = ?	UserPins	Frontend: ForumMapPage. jsx Backend: Server.js	Users can edit pins that they are the creator of.
19	Delete pin	DELETE FROM UserPins WHERE PinID = ? AND UserID = ?	UserPins	Frontend: ForumMapPage. jsx Backend: Server.js	Users can delete pins if they are the creator of the selected pin.
20	View reviews	SELECT r.*, u.UserName FROM PlaceReviews r JOIN Users u ON r.UserID = u.UserID WHERE r.LocationID = ? ORDER BY r.UpdatedAt DESC	PlaceReviews	Frontend: ForumMapPage. jsx Backend: Server.js	Users can view reviews when they select a pin.

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
21	Create review	INSERT INTO PlaceReviews (LocationID, UserID, Rating, ReviewText) VALUES (?, ?, ?, ?) ON DUPLICATE KEY UPDATE Rating = VALUES(Rating), ReviewText = VALUES(ReviewText)	PlaceReviews	Frontend: ForumMapPage.jsx Backend: Server.js	Users can create reviews for any pin on the map
22	Road status updates	SELECT IncludeStatus FROM Maps WHERE MapID = ?	Maps	Frontend: ForumMapPage.jsx Backend: Server.js	Users can view live road status updates if road statuses are included in the map
23	View legend	SELECT IncludeLegend FROM Maps WHERE ForumID = ? LIMIT 1	Maps	Frontend: ForumMapPage.jsx Backend: Server.js	Users can view the legend if the legend is included in the map
24	Edit Forum Tags	UPDATE Forums SET Tags = ? WHERE ForumID = ?	Forums	Frontend: ForumLandingPage.jsx Backend: Server.js	Tags used to categorize the forum.
25	Follow / Friend Users	//follow INSERT INTO UserRelationships (FollowingUser, FollowedUser)	UserRelationships Users	Frontend: UserProfilePage.jsx	Follow or unfollow user, different implementations are used for if the users are

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		VALUES (?, ?); //unfollow DELETE FROM UserRelationship s WHERE FollowingUser = ? AND FollowedUser = ? //friend INSERT INTO UserRelationship s (FollowingUser, FollowedUser, Relationship) VALUES (?, ?, "Friends"); UPDATE UserRelationship s SET Relationship = "Friends" WHERE FollowingUser = ? AND FollowedUser = ? DELETE FROM UserRelationship s WHERE FollowingUser =		Backend: Server.js	starting a mutual follow or breaking one.

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		? AND FollowedUser = ?; UPDATE UserRelationships SET Relationship = "Following" WHERE FollowingUser = ? AND FollowedUser = ?			
26	Block / Unblock Users	//block INSERT INTO UserRelationships (FollowingUser, FollowedUser, Relationship) VALUES (\${ReporterID}, \${ReporteeID}, "Blocked"); DELETE FROM UserRelationships WHERE (FollowingUser = \${ReporteeID} AND FollowedUser = \${ReporterID}) AND	Users, UserRelationships	Frontend: ProfilePage.jsx Backend: Server.js	Users can block another user so that user can no longer message them. This function is handled differently depending on if a relationship has already been created between users

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		(Relationship = "Friends" OR Relationship = "Following"); //block follower UPDATE UserRelationships SET Relationship = "Blocked" WHERE (FollowingUser = \${ReporterID} AND FollowedUser = \${ReporteeID}) DELETE FROM UserRelationships WHERE (FollowingUser = \${ReporteeID} AND FollowedUser = \${ReporterID}) AND (Relationship = "Friends" OR Relationship = "Following"); //unblock			

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		DELETE FROM UserRelationships WHERE (FollowingUser = \${ReporterID} AND FollowedUser = \${ReporteeID});			
27	Direct Messages	//view MessageID, SenderID, ReceiverID, MessageText, SentAt, UserName FROM Messages JOIN Users on SenderID = UserID WHERE (SenderID = ? AND ReceiverID = ?) OR (SenderID = ? AND ReceiverID = ?) ORDER BY SentAt ASC, MessageID ASC`, //post INSERT INTO Messages (SenderID,	Messages, User	Frontend: ProfilePage.jsx MessagesPage.js x Backend: Server.js	Users can view their messages to another user and post new messages

Implemented Use Cases + SQL Queries					
#	Use Case Name	SQL Query	Table(s)	File	Description
		ReceiverID, MessageText, SentAt) VALUES (?, ?, ?, NOW())			
28	Report Bug	INSERT INTO BugReports (Subject, SubmitterID) VALUES (?, ?)	BugReports	Frontend: ReportBugPage.js Backend: Server.js	Report bugs to the website staff
29	Thread Creation	INSERT INTO Threads (ThreadName, CategoryID, CreatorID) VALUES (?, ?, ?); SELECT LAST_INSERT_ ID() INTO @created_thread; INSERT INTO Posts (ThreadID, UserID, Content, OriginalThreadP ost) VALUES (@created_thread , ?, ?, 1);	Threads, Posts, Users	Frontend: ThreadContent.j sx, CategoryLandin gPage.jsx Backend: Server.js	Users can start a thread that other users can add posts onto
30	Report Content	INSERT INTO Reports (ReportType, Subject, SubmitterID, ReportedID) VALUES (?, ?, ?, ?)	Reports	Frontend: ReportPage.jsx Backend: Server.js	Report forums, threads, and posts to the website staff for removal.

*Table 3: Features and their SQL queries***System Sequence Diagrams*****Subsystem: Map - Chase Brown***

Figure 6 showcases how users interact with the map subsystem. Users can open the map, and the system will return the correct mapID along with the road statuses and user pins on that map. When the user right clicks anywhere on the map, they create a new pin by entering a title and description which is sent to the database to be stored. Once the database retrieves the data the map refreshes and displays the new user created pin. When a user selects a pin, the system retrieves and displays the information related to the pin the user selected. Users then have an opportunity to submit a review for the location the pin represents and the system will update and instantly display the new review. If the pin was created by the same user who selected it, the user will have the ability to delete the pin or edit the title, description, and visibility setting and the system will update or remove the pin accordingly.

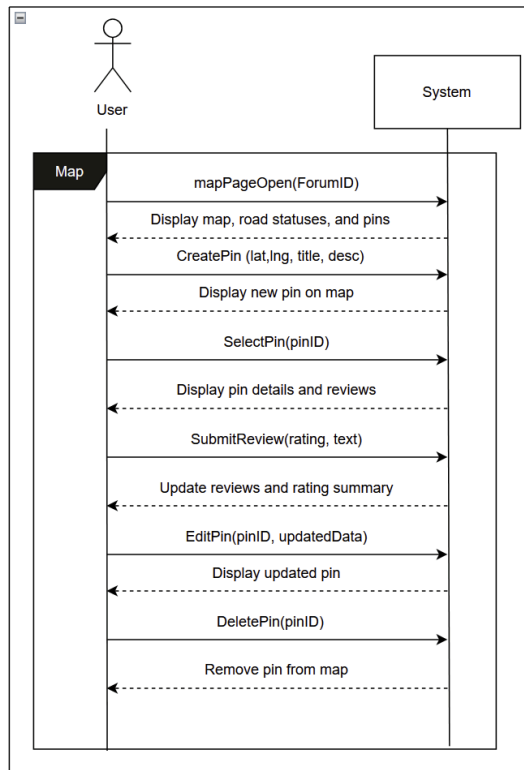


Figure 6: SSD for maps

Subsystem: Forum Interaction - Sean Kehoe

Figure 7 showcases how a user's interaction updates the database and receives feedback from the system. Editing allows a user to change the contents of their post. Creation allows a user to make a new post. Deletion removes the post's content and replaces it with a generic deletion message. Lastly, Rating Update allows users to rate a post and view the total likes and dislikes one has.

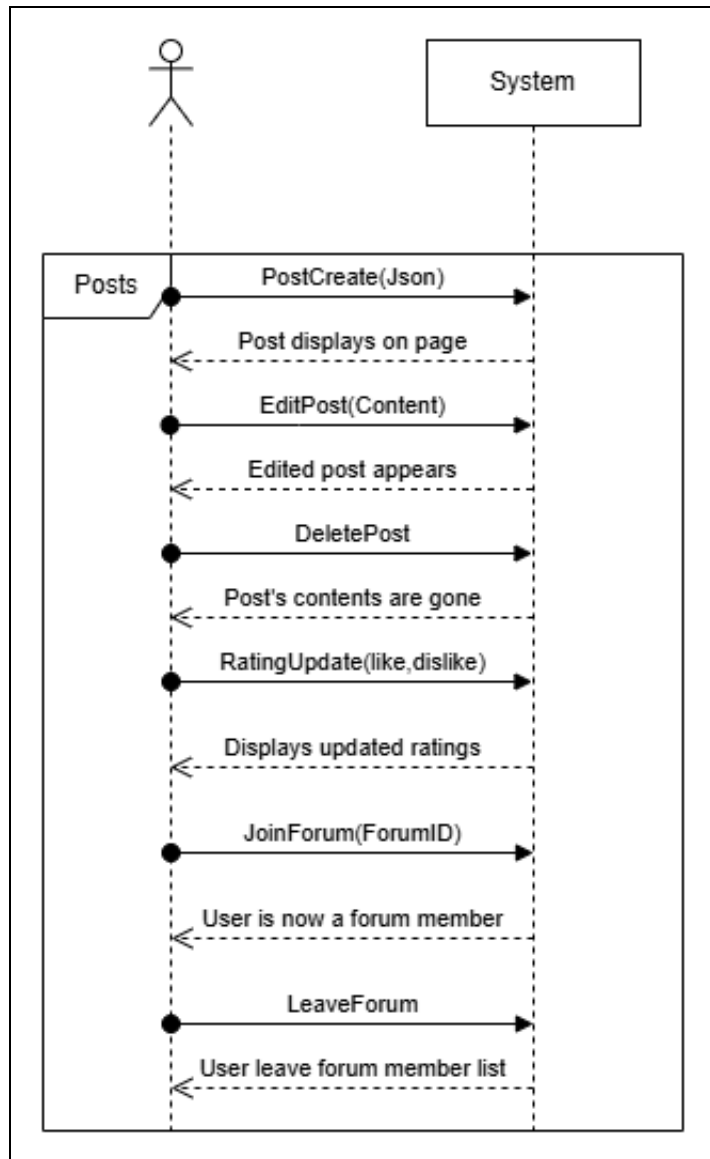


Figure 7: SSD For forum interaction

Subsystem: Website Interaction and Moderation - Jennifer Orlando

Figure 8 shows how the user is able to interact with the bug reporting interface. The user fills out a form detailing the bug and the form data is sent to the Reports table in the database.

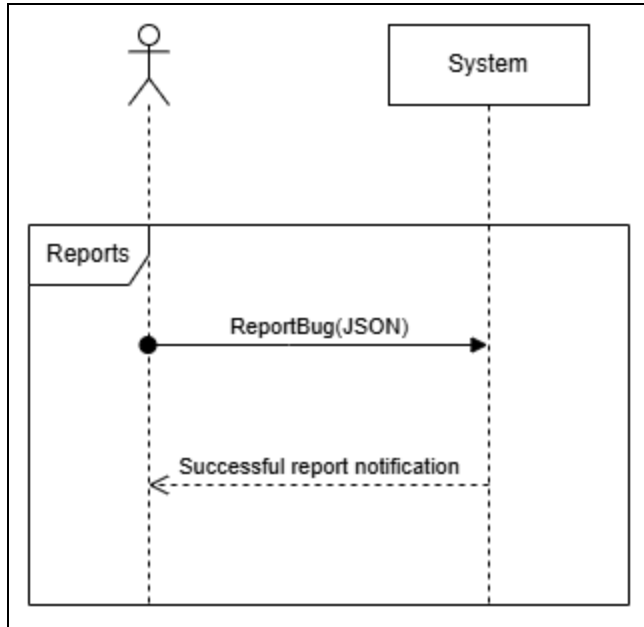


Figure 8: SSD for bug reporting

Subsystem: User Profiles and Direct Messaging - Mohammed Qudaih

Figure 9 showcases how users are able to view information about their profile through get requests to the website's database.

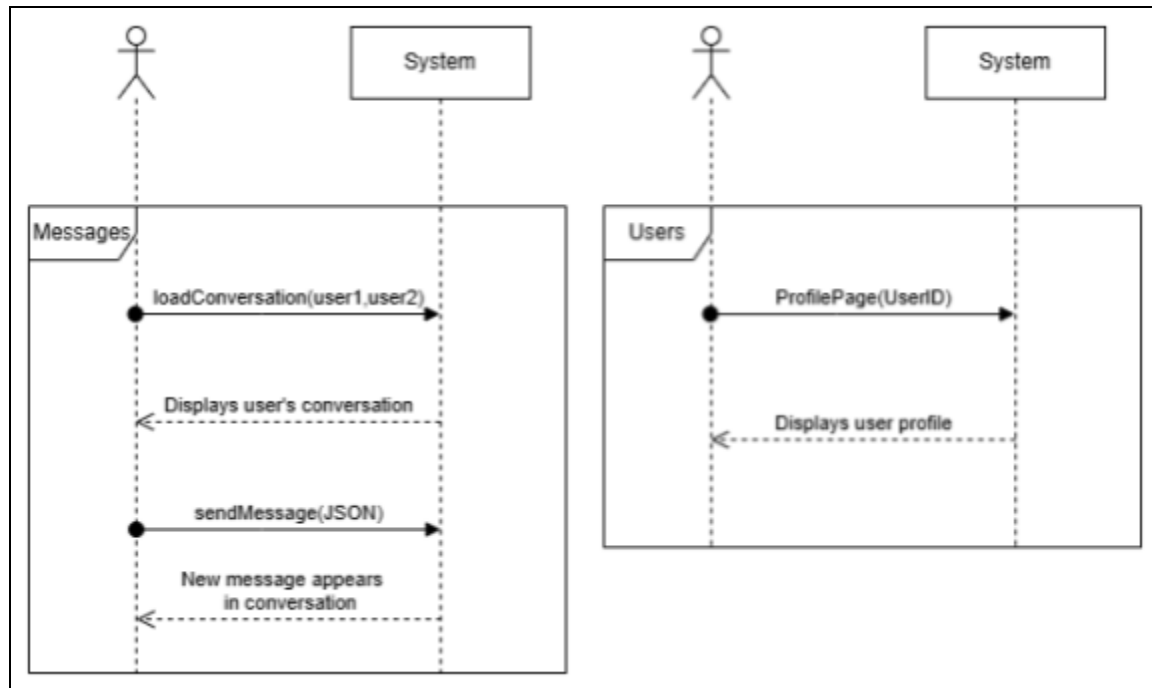
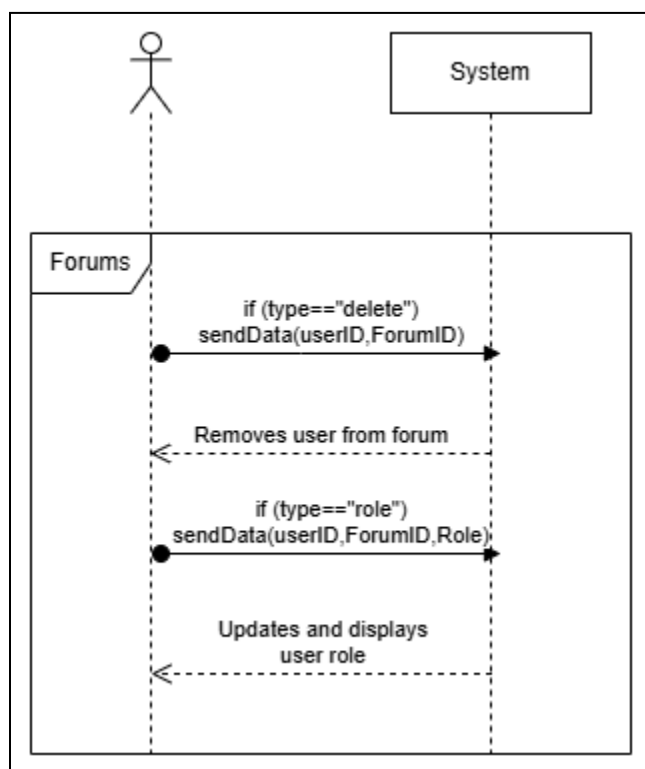
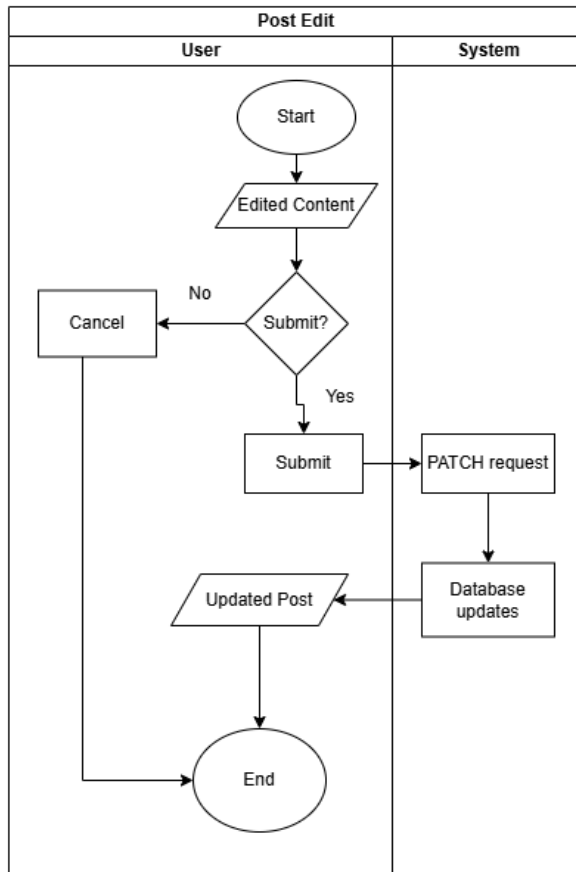


Figure 9: SSDs For user viewing their profile and user viewing their messages

Subsystem: Forums Admin Tools - Rory Strachan

Figure 10 illustrates how forum admins are able to alter the roles of user members. The User will fill out a form, the forum data will be sent to the backend, and the database will update the corresponding MemberList row.



*Figure 10: SSD for forum member role updating***Activity Diagram***Figure 11: Activity diagram for post editing*

User Interface Design

Wireframes

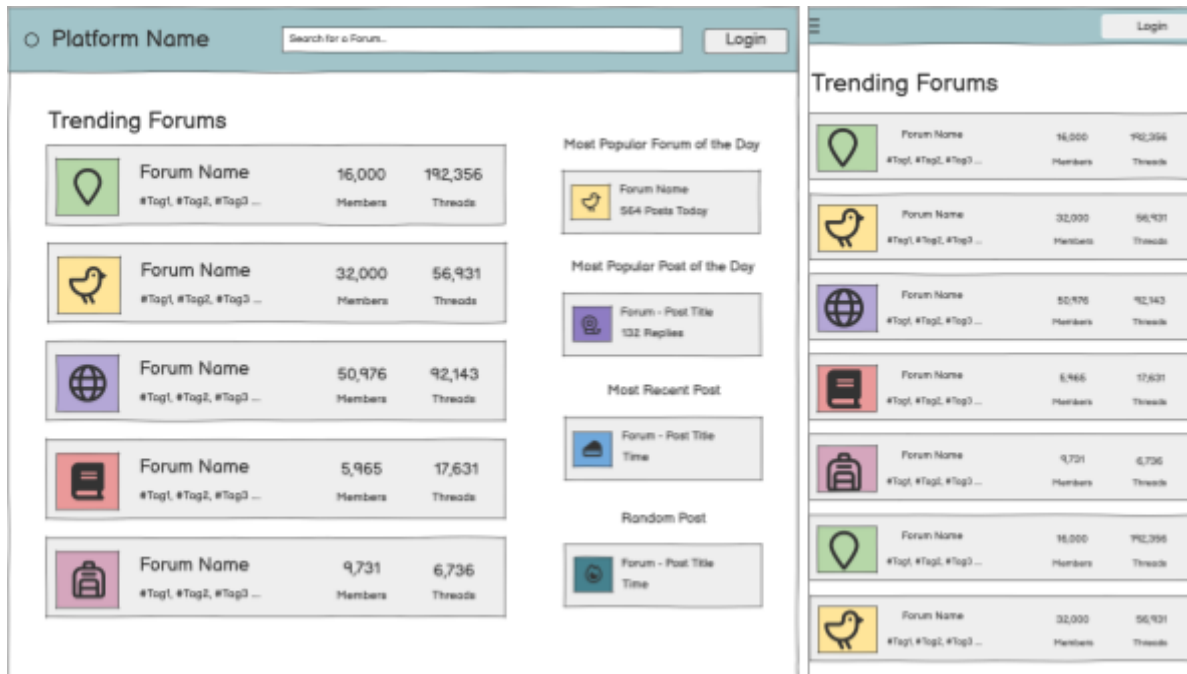


Figure 12: Wireframes of landing page

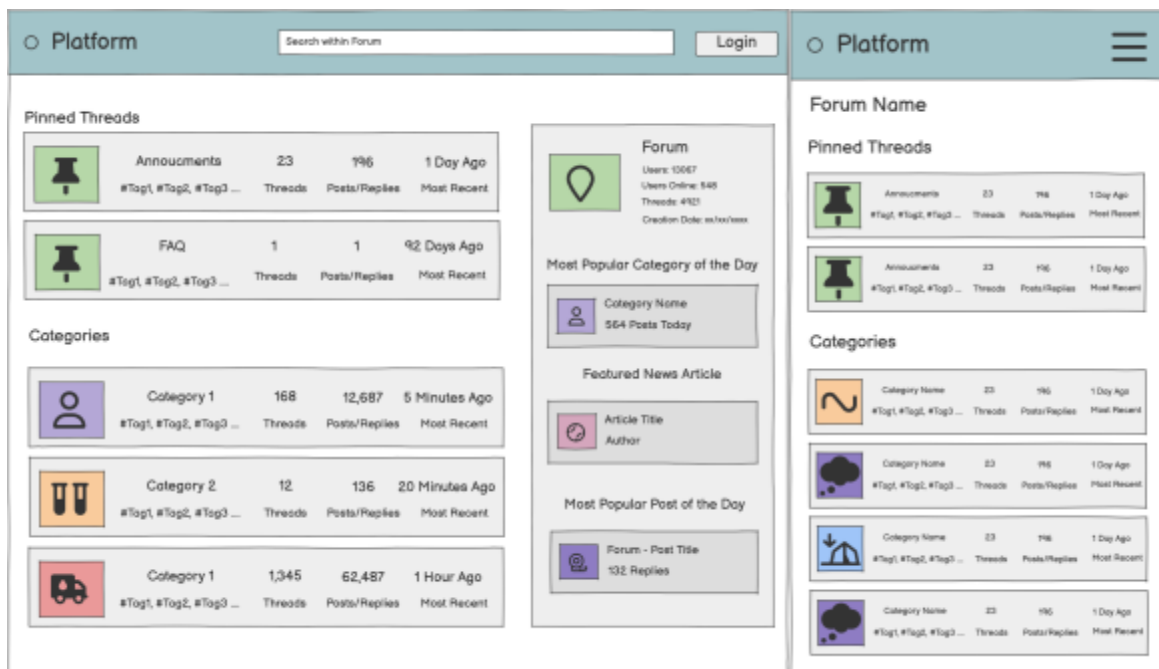


Figure 13: Wireframes of forum landing page

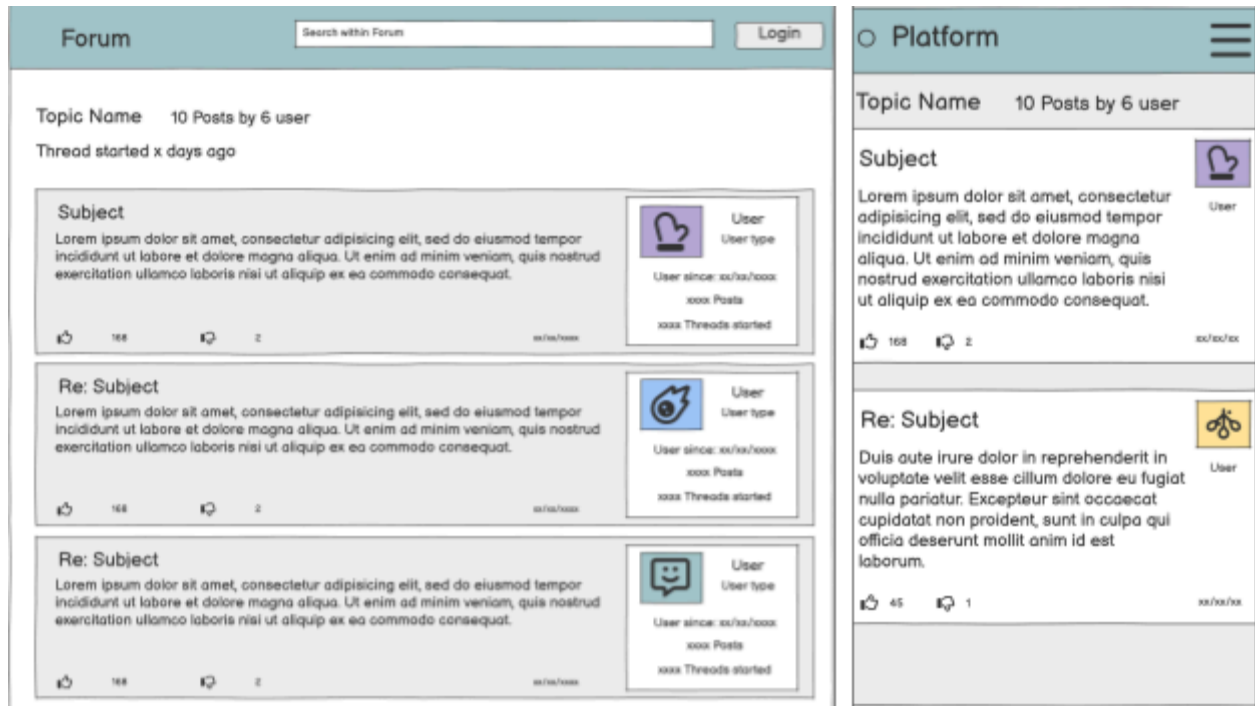


Figure 14: Wireframes of thread page

Mockups



Figure 15: Figma landing page mockup

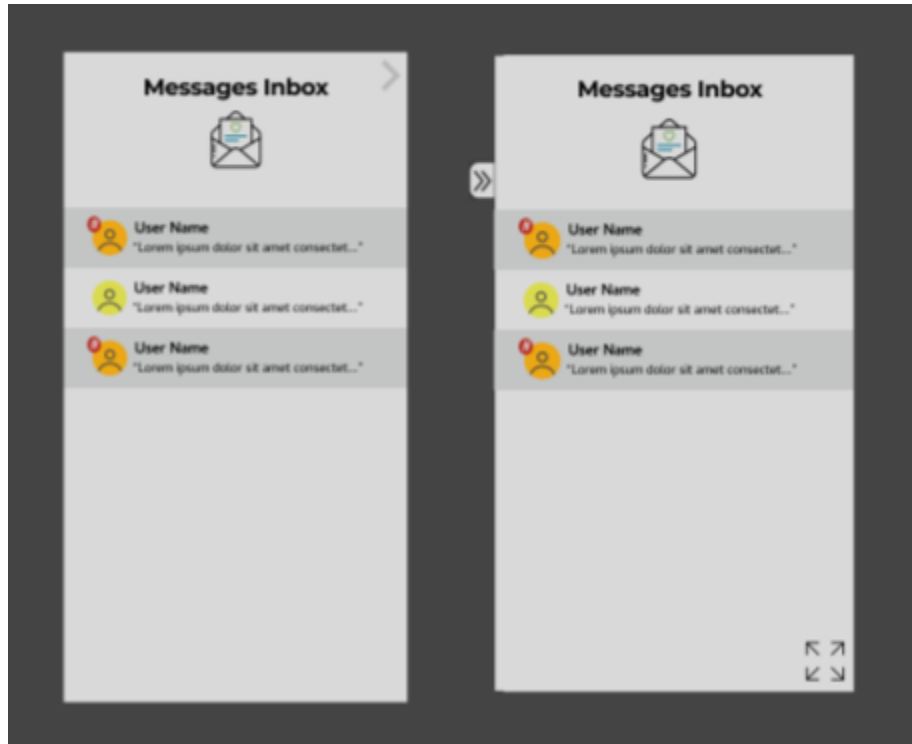


Figure 16: Figma messaging overlay mockup

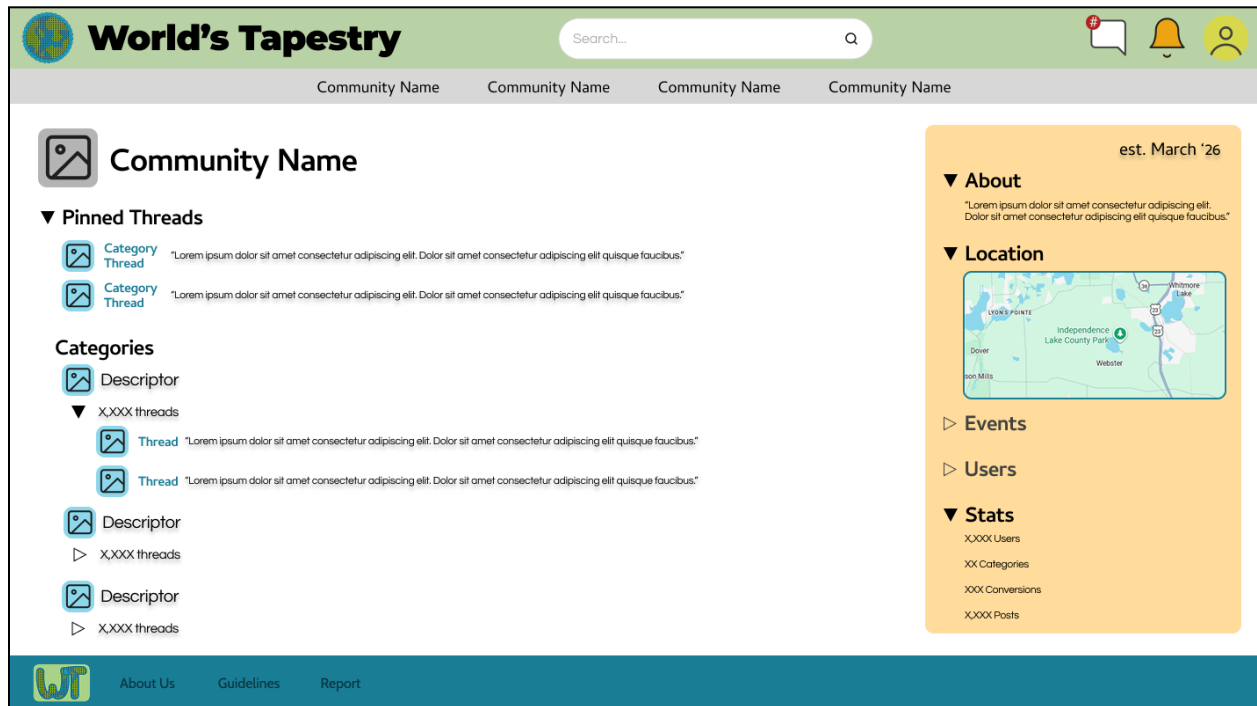


Figure 17: Forum landing page mockup

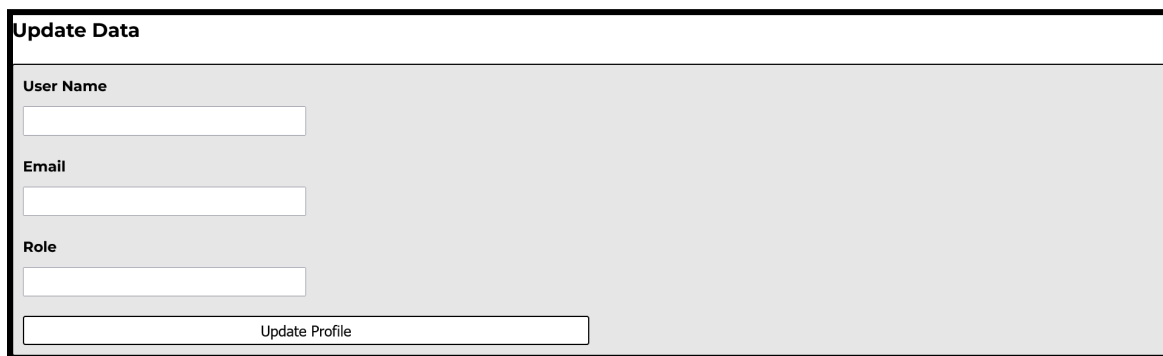
Responsive Design

Mobile versions have been created for all the designed wireframes and mockups. These versions trim down unnecessary detail and place greater focus on the main content of the webpage.

HCI Rules Discussion

Keep Users in Control

We utilized this principle across the entire site as users are in complete control of their experience by being able to visit threads at a click and are able to choose whether or not they wish to engage with a forum, threads, or other users. Additionally, the users do not need to worry about complex interfaces or controls as the site's simplicity allows them to clearly identify what their actions will do. A specific example of this would be our User info update feature which clearly and effectively spells out what it does.



The image shows a web form titled "Update Data". It has a light gray background and a black border. Inside the form, there are three input fields with labels "User Name", "Email", and "Role" above them. Below these fields is a button labeled "Update Profile".

Figure 18: A form within the web application

Permit Easy Reversal of Actions

This is utilized most prominently in our Profile update feature and Post editing features which allow users to change either personal information on their profiles or the contents of their posts to correct spelling mistakes or better clarify their positions. Furthermore, thanks to the

site's navigation tools a user can easily find their way back to the home page in order to get out of an unwanted destination such as a forum they find disagreeable.

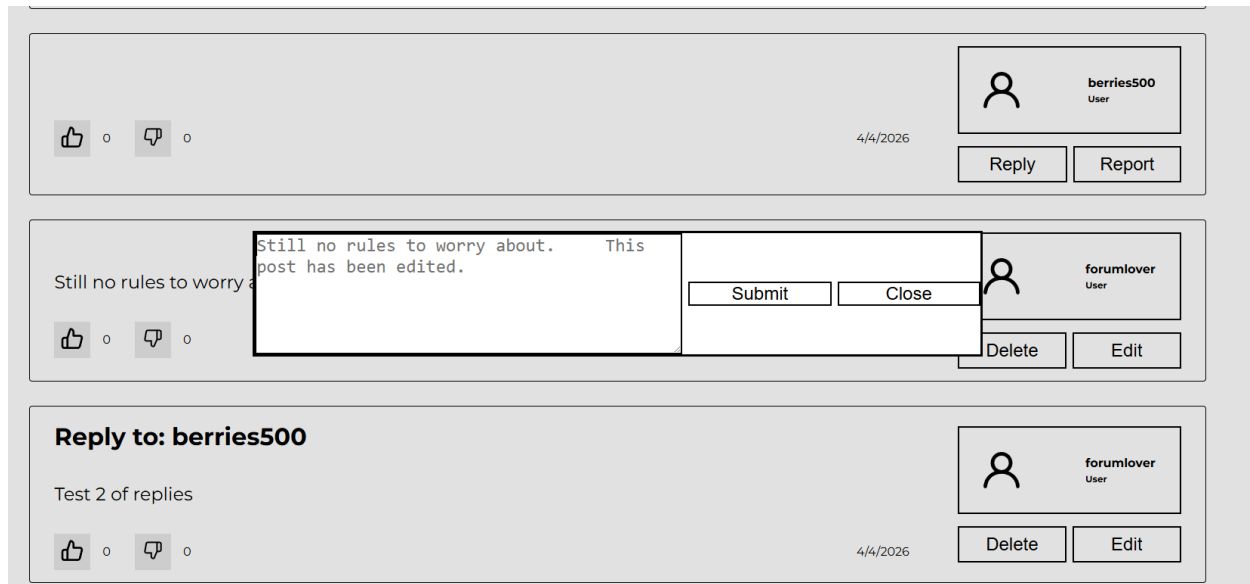
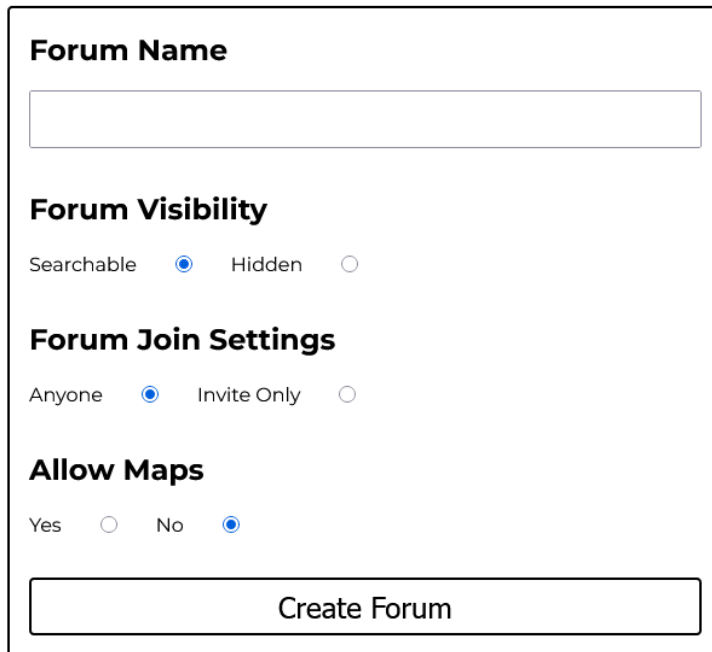


Figure 19: Post editing within the web application

Strive for Consistency

This is achieved in our site by maintaining a constant and consistent structure that allows users to readily identify what they are looking at when they are looking at it. For instance, our posts, threads, and forums all follow the same general layout allowing a user to easily identify the differences between each type of page that they are looking at while maintaining a uniform site styling that ensures that they have been directed to a page on our site.

Figure 20: Posts within the web application

Final Screenshots and Storyboard

The form is titled "Forum Name" and contains a text input field. Below this is the "Forum Visibility" section with two radio buttons: "Searchable" (selected) and "Hidden". The "Forum Join Settings" section has two radio buttons: "Anyone" (selected) and "Invite Only". The "Allow Maps" section has two radio buttons: "Yes" and "No" (selected). At the bottom is a "Create Forum" button.

Forum Name

Forum Visibility

Searchable ☒ Hidden ☐

Forum Join Settings

Anyone ☒ Invite Only ☐

Allow Maps

Yes ☐ No ☒

Create Forum

Figure 21: Forum Creation feature form

Forum creation feature where a user can enter a forum name, set the forum's visibility and join settings, and choose whether they wish to have a map or not.

Admin Dash

Forum Settings

Add New Category

Category Name

Category Description

Pin Status
Pinned ☐ Not Pinned ☒

Create Category

Figure 22: Category Creation feature form

Category creation feature where a user is able to create a brand new category inside of a forum by inputting its name and a short description so people can easily determine what it is about, and can set it as a pinned category in the forum to elevate its status and position on screen.

Username: ChaseTest	Role: Member ▾	Remove Member ☐	Submit
Username: DerekLutz	Role: Admin ▾	Remove Member ☐	Submit
Username: LorettaMuring	Role: Admin ▾	Remove Member ☐	Submit
Username: MarkLowe	Role: Member ▾	Remove Member ☐	Submit
Username: JacobSchmeling	Role: Member ▾	Remove Member ☐	Submit
Username: JohannaSmith	Role: Member ▾	Remove Member ☐	Submit

Figure 23: User Removal and Role change form

User role form, a dual purpose form that can alter any given user's role inside of a forum alongside choosing whether or not a user should be removed from the forum's member list. This allows the use of the user removal feature and user role assignment in one location.

 **Basketball**

Leave Forum

Top Tags

Figure 24: Forum join/leave feature

Forum join/leave feature that allows users to join or leave a forum by clicking on the respective button which will cause them to be added to the forum's member list.



Figure 25: Post Like/Dislike feature

Post like/dislike feature using a brand new test post to showcase a post with no likes or dislikes. Clicking on either icon will raise the amount of likes or dislikes respective to the button clicked and currently has an either/or restriction set in place to prevent a user from both liking and disliking a post and only allowing one per user.

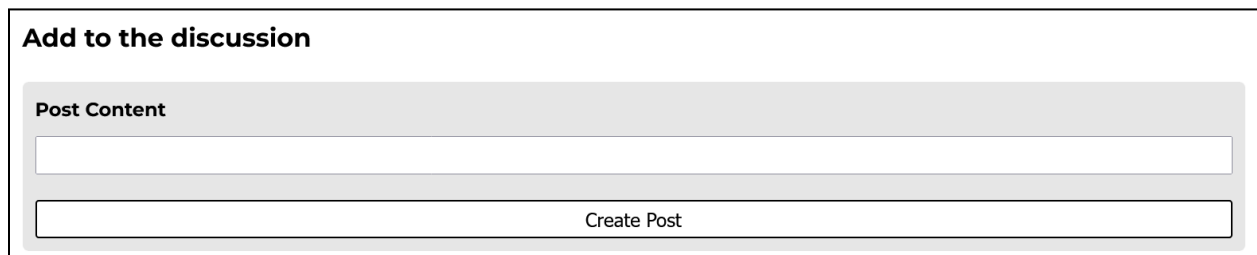
A screenshot of a form titled "Add to the discussion". Inside the form, there is a section labeled "Post Content" which contains two text input fields. The first field is empty, and the second field is also empty. Below the second input field, there is a button labeled "Create Post".

Figure 26: Post Creation feature

Post Creation feature which allows a user, based on posting privileges, the ability to create a new post inside of a thread on the forum. They do so by creating a subject and then write in the body of the post in the second field. Once they are finished they will then hit submit which will send the newly constructed post to the database and refresh the page displaying the new post on the thread.



Figure 27: Post Editing Feature

Post editing feature displaying the current working editing field that was called after clicking the edit button on the first post. In here the user will enter whatever changes they wish to their post inside the text field on the left hand side of the modal and can then choose to click submit which will finalize changes and send the new data to the backend thus changing the post or can click close which will cause the form to close and will not result in a change being made.

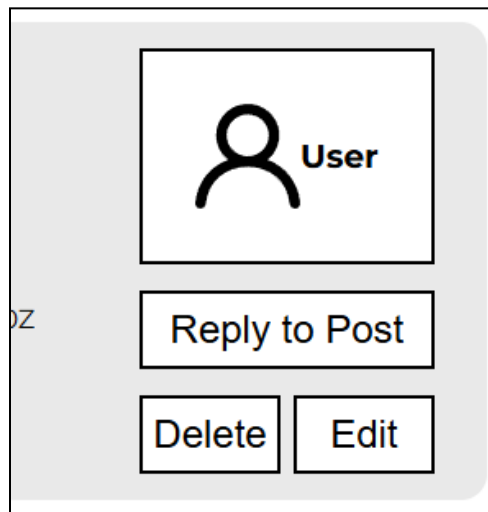


Figure 28: Post Deletion feature

Post delete feature which is used by clicking the delete button shown on screen below the User info field. This will result in the contents of the user's post being replaced with "This post has been deleted" thus removing the previous content from the database and site.

ChangeMe1



ChangedAgain@xxx.com

Following:

0

Followers:

0

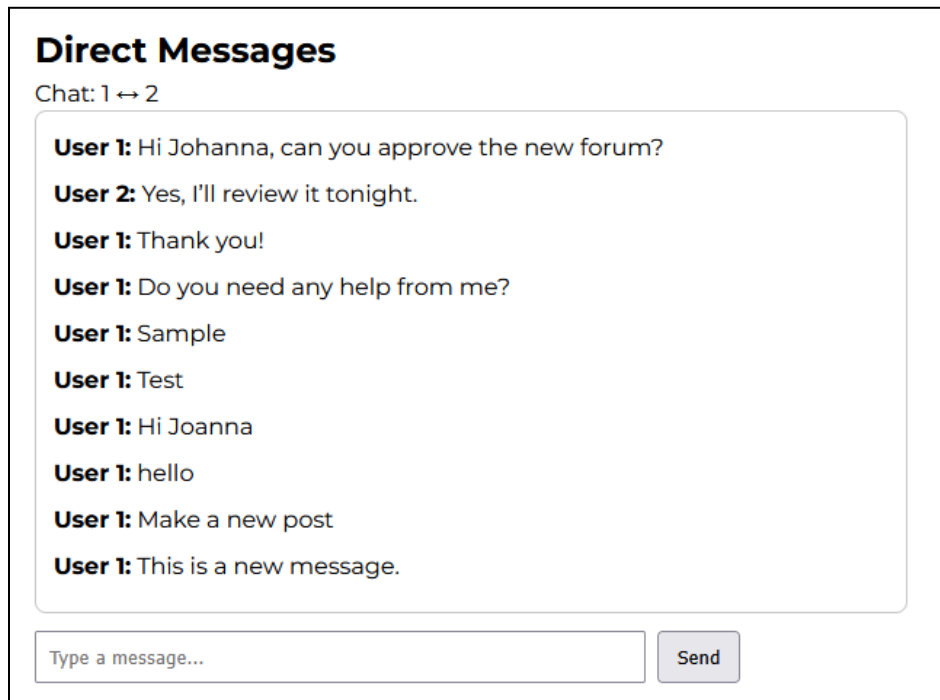
Figure 29: User Profile showing name, image, email and follow/following counts

User profile that allows a user to see their current user name, ID, email, role, and the date the profile was created.

Update Data	
User Name	
Email	
Role	
Update Profile	

Figure 30: User Profile Update feature

User update feature that allows a user to change their user name, email, or role, by entering the new data into their respective fields and then clicking the update profile button to submit changes to the database.



The screenshot shows a web interface for direct messaging. At the top, the title "Direct Messages" is displayed in bold. Below it, the chat header reads "Chat: 1 ↔ 2". The main area is a list of messages, all attributed to "User 1":

- Hi Johanna, can you approve the new forum?
- User 2: Yes, I'll review it tonight.
- Thank you!
- Do you need any help from me?
- Sample
- Test
- Hi Joanna
- hello
- Make a new post
- This is a new message.

At the bottom, there is a text input field with the placeholder "Type a message..." and a "Send" button to its right.

Figure 31: Direct Messaging Feature

Direct messaging feature that allows a user to view their pre-existing messages with another user while also giving them the ability to send a new message to the same or other users.

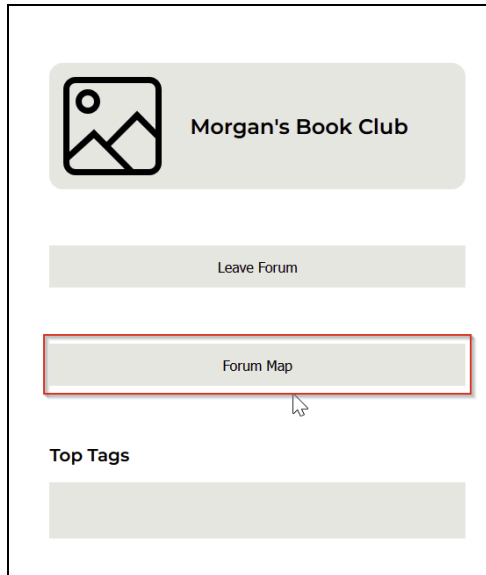


Figure 32: Navigating to the forum map

The user navigates to a forum that has the map feature enabled. On the right hand side of the page, they can select the “Forum Map” button to be redirected to the map page.

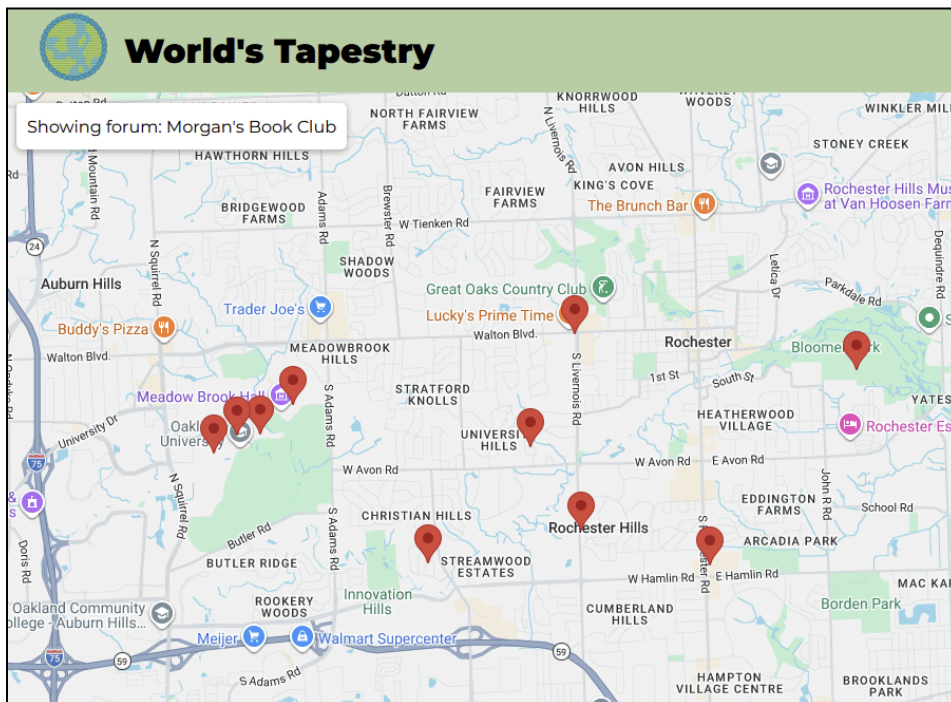


Figure 33: The forum map

Upon viewing the map, the user will see existing businesses and user created pins.

When the user right clicks on the map, they can enter basic pin information such as a title, description, and set their visibility setting. Once they click the “Place Pin” button, it will appear on the map. If the user clicks on the “Delete Pin” button, the pin will be removed from the map.

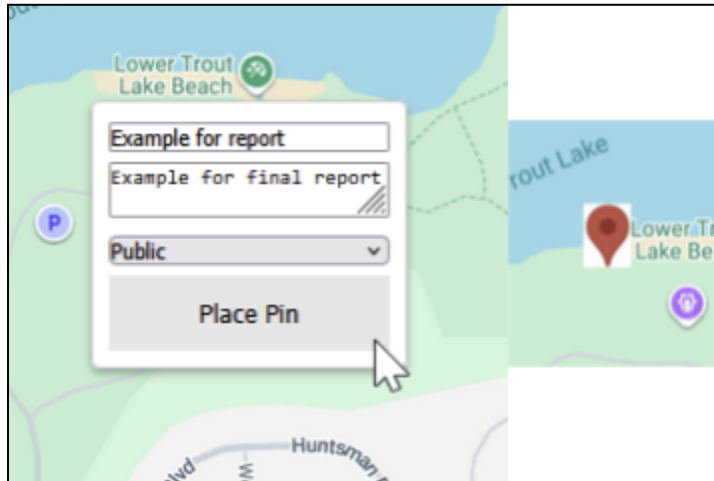


Figure 34: Creating a pin

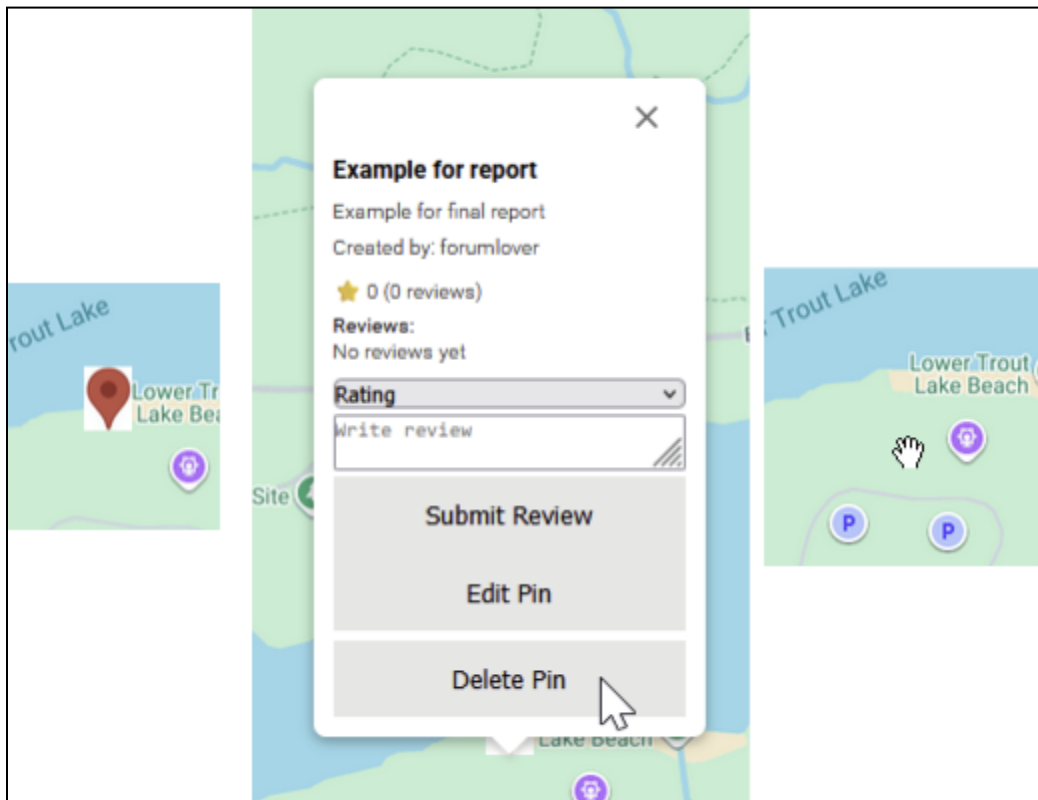


Figure 35: Deleting pin points

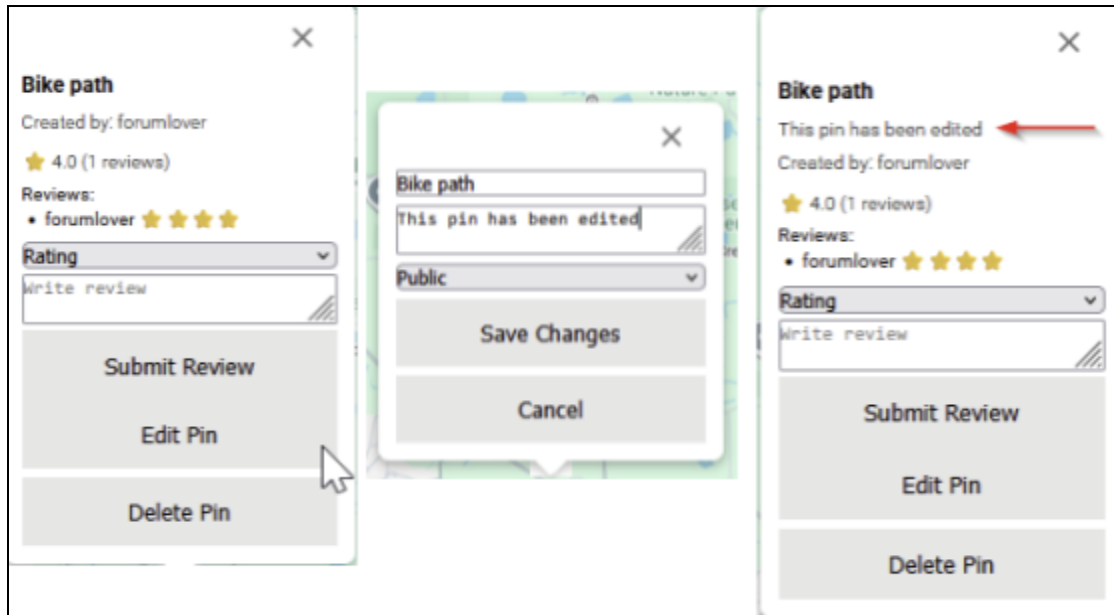


Figure 36: Editing a pin

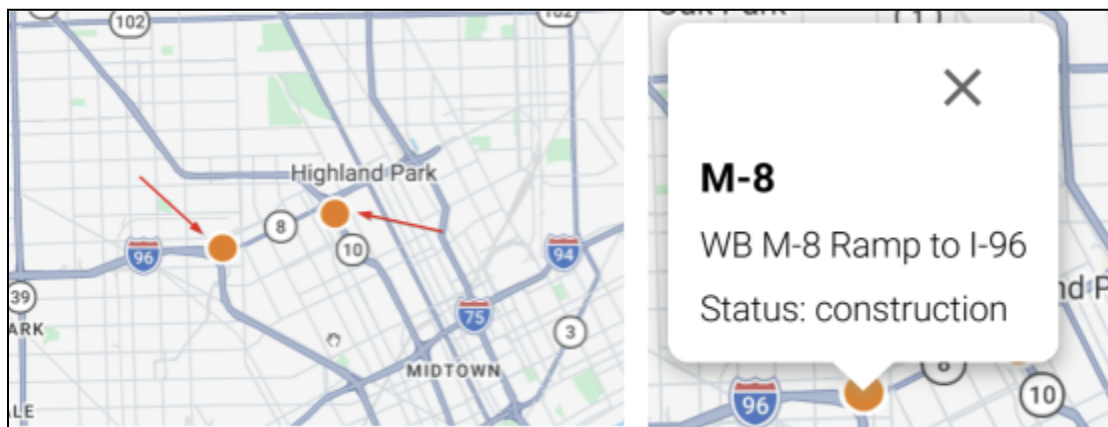


Figure x:

Figure 37: Viewing live road status updates

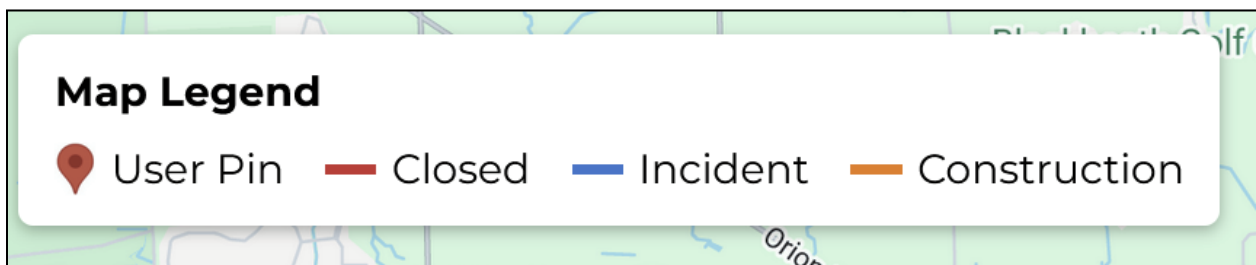


Figure 38: Viewing the map legend

Database Design

All Tables

▶	BugReports
▶	Categories
▶	Forums
▶	Locations
▶	Maps
▶	MemberList
▶	Messages
▶	PlaceReviews
▶	PostRatings
▶	Posts
▶	Reports
▶	Threads
▶	UserPins
▶	UserRelationships
▶	Users

Figure 39: Forums table

Table: Forums (Main table)

ForumID	ForumName	CreationDate	MemberCount	Tags	SearchVisibility	JoinPermissions	AllowMaps	OwnerID	MostRecentActivity
1	Book Club	2026-03-25 04:02:03	1	NULL	Searchable	Anyone	0	1	NULL
2	Game club	2026-03-25 04:02:32	1	NULL	Searchable	Anyone	0	8	NULL
16	Rochester Biking	2026-03-25 04:06:40	1	NULL	Searchable	Anyone	1	5	NULL
17	Movie Review Hub	2026-03-25 04:09:07	1	NULL	Searchable	Anyone	1	2	NULL
31	Stoney Creek Nature Club	2026-03-25 05:31:12	1	NULL	Searchable	Anyone	1	1	NULL
32	Rochester Camping	2026-03-25 13:06:25	1	NULL	Searchable	Anyone	1	1	NULL
33	Food and Dining Guide	2026-03-25 17:10:54	1	NULL	Searchable	Anyone	1	2	NULL
34	Food and Dining Guide	2026-03-25 17:11:29	1	NULL	Searchable	Anyone	1	1	NULL
35	Paint Creek Trail and Outdoor Activities	2026-03-25 17:12:05	1	NULL	Searchable	Anyone	1	1	NULL
36	Rochester Events	2026-03-25 17:12:31	1	NULL	Searchable	Anyone	0	1	NULL
37	Local Shopping	2026-03-25 17:12:41	1	NULL	Searchable	Anyone	0	1	NULL
38	Nightlife and Entertainment	2026-03-25 17:12:53	1	NULL	Searchable	Anyone	1	1	NULL
39	Parks, Playgrounds, and Family Spots	2026-03-25 17:13:09	1	NULL	Searchable	Anyone	1	1	NULL
40	Study Spots	2026-03-25 17:13:35	1	NULL	Searchable	Anyone	1	1	NULL
41	Best Coffee Shops	2026-03-25 17:13:56	1	NULL	Searchable	Anyone	1	1	NULL

Figure 40: Forums table

Table: Categories

CategoryID	ForumID	CategoryName	CategoryDescription	Pinned	NumberOfThreads	NumberOfPosts	MostRecentActivity
2	2	Rules	Please read before posting	1	1	1	2026-03-25 04:54:39
3	2	General	Talk and hang out!	0	1	1	2026-03-28 00:39:57
4	2	Monthly Game Discussion	We pick a new game each month... Come here ...	0	2	4	2026-03-25 04:58:01
6	1	Welcome	Rules etc	0	7	7	2026-03-25 16:54:22
7	31	Welcome	Who we are.. and other things	1	1	1	2026-03-25 05:31:38
8	16	Best Bike Parts	What are you riding?	0	1	2	2026-03-28 00:42:07
9	16	Best Trails	Where are your favorite spots?	0	1	1	2026-03-28 00:41:37
10	32	Best Campgrounds	Where do you and your family camp?	0	1	1	2026-03-28 00:43:19
11	32	Camping Foods	What's in the cooler?	0	1	1	2026-03-28 00:44:57
12	40	Quiet places	Places to study outside of Oakland University	0	1	1	2026-03-28 00:45:43

Figure 41: Categories table

Table: BugReport

	BugReportID	Subject	SubmitterID
▶	1	sample	1
	2	Can't complain at a person I hate	1
	3	Map has been glitching out and deleting my pi	1
	4	Can't see another person's pins	1
	5	Couldn't report someone for a bannable offenses	1
	6	This site sucks and I hate you all	1
	7	Found a user somehow able to post pictures wh	1
*	NULL	NULL	NULL

*Figure 42: Bug Report table***Table: Locations**

	LocationID	LocationName	Latitude	Longitude
▶	1	Another great trail	42.719658	-83.166246
	2	Trail	42.718421	-83.163843
	3	Really cool spot by the river	42.743261	-83.269758
	4	Cool river spot	42.747221	-83.226509
	5	cool river trail	42.743639	-83.109083
	6	cool river spot	42.742607	-83.103761
	7	cool river spot	42.742871	-83.103091
	8	cool river spot	42.743237	-83.107194
	9	another great spot	42.746161	-83.149423
	10	Bike path	42.749060	-83.183755

*Figure 43: Locations table***Table: Maps**

	MapID	ForumID	IncludeStatus	IncludeLegend
	2	16	1	1
	3	31	1	1
	4	32	1	1
	5	33	1	1
	6	34	1	1
	7	35	1	1
	8	38	1	1
	9	39	1	1
	10	40	1	1
	11	41	1	1

Figure 44: Maps table

Table: MemberList

	ForumID	UserID	UserRole
▶	1	0	Member
	1	1	Owner
	2	1	Admin
	2	8	Owner
	16	1	Admin
	16	5	Owner
	17	1	Member
	17	2	Owner
	31	1	Owner
	32	1	Owner
	32	2	Member
	33	2	Owner
	34	1	Owner
	35	1	Owner
	36	1	Owner
	37	1	Owner
	38	1	Owner

Result 14 MemberList 15 ×

*Figure 45: MemberList table***Table: Messages**

	MessageID	SenderID	ReceiverID	MessageText	SentAt
▶	1	2	1	Text 1	2026-04-04 04:24:18
	2	4	1	Text 2	2026-04-04 04:24:18
	3	1	3	Text 3	2026-04-04 04:24:18
	4	1	5	Text 4	2026-04-04 04:24:18
	5	5	3	Text 5	2026-04-04 04:24:52
	6	3	6	Text 6	2026-04-04 04:24:52
	7	1	2	Text 7	2026-04-04 04:26:38
	8	2	1	Text 8	2026-04-04 04:26:39
	9	1	2	Sample message	2026-04-04 06:46:42
	10	1	4	test	2026-04-05 17:20:04
	11	1	4	okay	2026-04-05 17:20:19
	12	1	10	hi	2026-04-05 17:21:51
	13	1	2	Hey man can y...	2026-04-06 00:30:53
	14	1	4	Why only 700 fl...	2026-04-06 00:53:59
*	NULL	NULL	NULL	NULL	NULL

Result 14 Messages 15 ×

Figure 46: Messages table

Table: PlaceReviews

	ReviewID	LocationID	UserID	Rating	ReviewText	CreatedAt	UpdatedAt
▶	1	2	8	3	Just a trail, nothing special	2026-03-28 18:09:48	2026-03-28 18:09:48
	2	8	1	5		2026-03-28 19:13:27	2026-03-28 19:13:27
	3	9	8	3		2026-03-28 19:53:56	2026-03-28 19:53:56
	4	10	1	4		2026-03-28 20:07:46	2026-03-28 20:07:46
	5	2	1	5	Just went on a ride here. It was a blast!	2026-03-29 14:38:33	2026-03-29 14:40:43

*Figure 47: PlaceReviews table***Table: PostRatings**

	PostID	UserID	Rating
▶	20	1	0
	29	1	0
	30	1	0
	31	1	0
	33	1	0
	34	1	0
	35	1	0
*	NULL	NULL	NULL

*Figure 48: PostRatings table***Table: Posts**

Result Grid										
Filter Rows:				Edit:		Export/Import:		Wrap Cell Content:		
	PostID	ThreadID	UserID	Content	Likes	Dislikes	Subject	Created_Time	OriginalThreadPost	
▶	14	11	2	January: Ao Oni!!!	0	0		2026-01-01 04:50:28	1	0
	15	11	2	Febuary: Incredible Crisis	0	0		2026-02-02 04:51:21	0	0
	16	11	4	March: Blues Brothers 2000	0	0		2026-03-25 04:52:47	0	0
	17	12	1	Rule 1) Have fun :)	0	0		2026-03-25 04:54:39	1	0
	18	13	1	How do we help him be happy!	0	0		2026-03-25 04:58:01	1	0
	20	21	1	There are no major rules to the forum. This p...	1	0		2026-03-25 05:09:50	1	0
	21	22	1	We are a book club.	0	0		2026-03-25 05:10:45	1	0
	22	23	1	This is a thread for posting off-topic discussions...	0	0		2026-03-25 05:11:31	1	0
	23	24	1	yeah	0	0		2026-03-25 05:11:52	1	0
	24	25	1	Hello	0	0		2026-03-25 05:31:38	1	0
	25	26	1	1984 by George Orwell	0	0		2026-03-25 16:46:56	1	0
	26	27	1	Theres a new thread here	0	0		2026-03-25 16:51:45	1	0
	27	28	1	demo thread	0	0		2026-03-25 16:54:22	1	0
	28	29	1	Drop your favorites here!	0	0		2026-03-28 00:39:57	1	0
	29	30	1	Where is everyone buying their parts?	1	0		2026-03-28 00:40:56	1	0
	30	31	1	I really like biking around the University!	1	0		2026-03-28 00:41:37	1	0

Figure 49: Posts table

Table: Reports

	ReportID	ReportType	Subject	SubmitterID	ReportedID
▶	2	Forum	sample forum report	1	1
	3	Thread	sample thread report	1	21
	4	Post	sample post report	1	35
	5	Forum	Sampel report for book club	1	1
	6	Post	Sample post report	1	35
	7	Post	Test 2 of replies	1	35
	8	Thread	This thread sucks eggs.	1	21
	9	Forum	This forum has been nothing but stale content.	1	1
✱	NULL	NULL	NULL	NULL	NULL

*Figure 50: Reports table***Table: Threads**

	ThreadID	ThreadName	CategoryID	CreatorID	Locked	WhoCanPost	DateCreated	MostRecentActivity	NumberOfPosts
▶	11	Monthly Picks	4	1	0	"Anyone"	2026-03-25 04:50:28	2026-04-05 17:21:17	4
	12	REQUIRED READING	2	1	0	"Anyone"	2026-03-25 04:54:39	2026-03-25 04:54:39	1
	13	Discussion on Ao Oni	4	1	0	"Anyone"	2026-03-25 04:58:01	2026-03-25 04:58:01	1
	21	RULES	6	1	0	"Anyone"	2026-03-25 05:09:50	2026-04-06 00:56:49	8
	22	Who we are	6	1	0	"Anyone"	2026-03-25 05:10:45	2026-03-25 05:10:45	1
	23	Etc	6	1	0	"Anyone"	2026-03-25 05:11:31	2026-03-25 05:11:31	1
	24	final test	6	1	0	"Anyone"	2026-03-25 05:11:52	2026-03-25 05:11:52	1
	25	Welcome	7	1	0	"Anyone"	2026-03-25 05:31:38	2026-03-25 05:31:38	1
	26	Our Favorite Books	6	1	0	"Anyone"	2026-03-25 16:46:56	2026-03-25 16:46:56	1
	27	New thread	6	1	0	"Anyone"	2026-03-25 16:51:45	2026-03-25 16:51:45	1
	28	New thread example	6	1	0	"Anyone"	2026-03-25 16:54:22	2026-03-25 16:54:22	1
	29	Looking for new ga...	3	1	0	"Anyone"	2026-03-28 00:39:57	2026-03-28 00:39:57	1
	30	Local shops	8	1	0	"Anyone"	2026-03-28 00:40:56	2026-03-28 00:42:07	2
	31	Rochester	9	1	0	"Anyone"	2026-03-28 00:41:37	2026-03-28 00:41:37	1
	32	Show us your camp...	10	1	0	"Anyone"	2026-03-28 00:43:19	2026-03-28 00:43:19	1
	33	S'more lovers	11	1	0	"Anyone"	2026-03-28 00:44:57	2026-03-28 00:44:57	1
	34	Starbucks	12	1	0	"Anyone"	2026-03-28 00:45:43	2026-03-28 00:45:43	1

*Figure 51: Threads table***Table: UserPins**

PinID	UserID	Visibility	Longitude	Latitude	Title	Description	MapID	LocationID
9	1	Public	-83.20383974609373	42.66422163405743	camping site		4	11
11	8	Public	-83.25199848537646	42.66545355808585	Fishing Dock	Good spot for early morning fishing.	4	12
13	8	Public	-83.20555635986327	42.662214559400354	Bike trail	Great nature trail	2	13
14	8	Public	-83.12896661729971	42.69676271369938	Monday ride Meetup	Meet here by 5:00pm to ride with us!	2	14
15	8	Public	-83.10350212193492	42.71866441502183	Race	Race around the golf course!	2	15
16	8	Public	-83.20845074682487	42.72068243361649	Relaxing ride	Great spot for a casual bike ride	2	16
17	8	Public	-83.21514554052604	42.672248219624585	University ride	Biking around campus is my favorite. This pin ha...	2	17
18	8	Public	-83.10501969381957	42.66767097117567	Very pretty homes	I love cycling around and looking at these nice h...	2	18
19	8	Public	-83.3400241188684	42.631434668148174	Lake spot	Nice ride through the neighborhood that ends w...	2	19
20	8	Public	-83.36028016134887	42.70413845947999	Lake Oakland	Lake Oakland is so pretty!	2	20
21	8	Public	-83.166245904541	42.71965774548897	Another great trail	Lots of hills	2	1
22	8	Public	-83.16384264526366	42.71842075968203	Trail		2	2
23	1	Public	-83.26975771484373	42.74326123188466	Really cool spot by ...		2	3
24	1	Public	-83.22650869483394	42.74722127221952	Cool river spot		2	4
25	1	Public	-83.10908266601561	42.74363943628213	cool river trail		2	5
26	1	Public	-83.10376116333006	42.74260654849799	cool river spot		2	6
27	1	Public	-83.10309051216677	42.74287127882314	cool river spot		2	7
28	1	Public	-83.10719439086913	42.74323689453501	cool river spot		2	8
29	8	Public	-83.1494230895996	42.74616073996991	another great spot		2	9
30	1	Public	-83.18375536499022	42.74906011244175	Bike path	This pin has been edited	2	10

*Figure 52: UserPins table***Table: UserRelationships**

	FollowingUser	FollowedUser	RelationStarted	Relationship
▶	1	5	2026-04-05 17:45:30	Blocked
*	NULL	NULL	NULL	NULL

*Figure 53: UserRelationships table***Table: Users**

UserID	UserName	Email	CreationDate	FollowerCount	FollowingCount	Password	ProfilePicture
1	forumlover	user@gmail.com	2026-03-25 03:58:59	0	0	1	avatar3.png
2	berries500	berry@gmail.com	2026-03-25 03:59:18	0	0	1	avatar4.png
3	xx_xx	tshirt@gmail.com	2026-03-25 04:01:36	0	0	1	avatar1.png
4	700flowers	berry2@gmail.com	2026-03-25 04:01:36	0	0	1	avatar2.png
5	metalmario	metal@gmail.com	2026-03-25 04:01:36	0	0	1	avatar10.png
6	ILOVECOMPUTER	pc@gmail.com	2026-03-25 04:01:36	0	0	1	avatar6.png
7	IHATECOMPUTER	nopc@gmail.com	2026-03-25 04:01:36	0	0	1	avatar8.png
8	playstation7	ps7@gmail.com	2026-03-25 04:01:36	0	0	1	avatar5.png
9	segasaturn	saturn@gmail.com	2026-03-25 04:01:36	0	0	1	avatar7.png
10	laurence	laurence@gmail.com	2026-03-25 04:01:36	0	0	1	avatar9.png
11	MESSAGEFORFR...	scam@gmail.com	2026-03-25 04:01:36	0	0	1	avatar7.png

Figure 54: Users table

Entity Relationship Diagram (ERD)

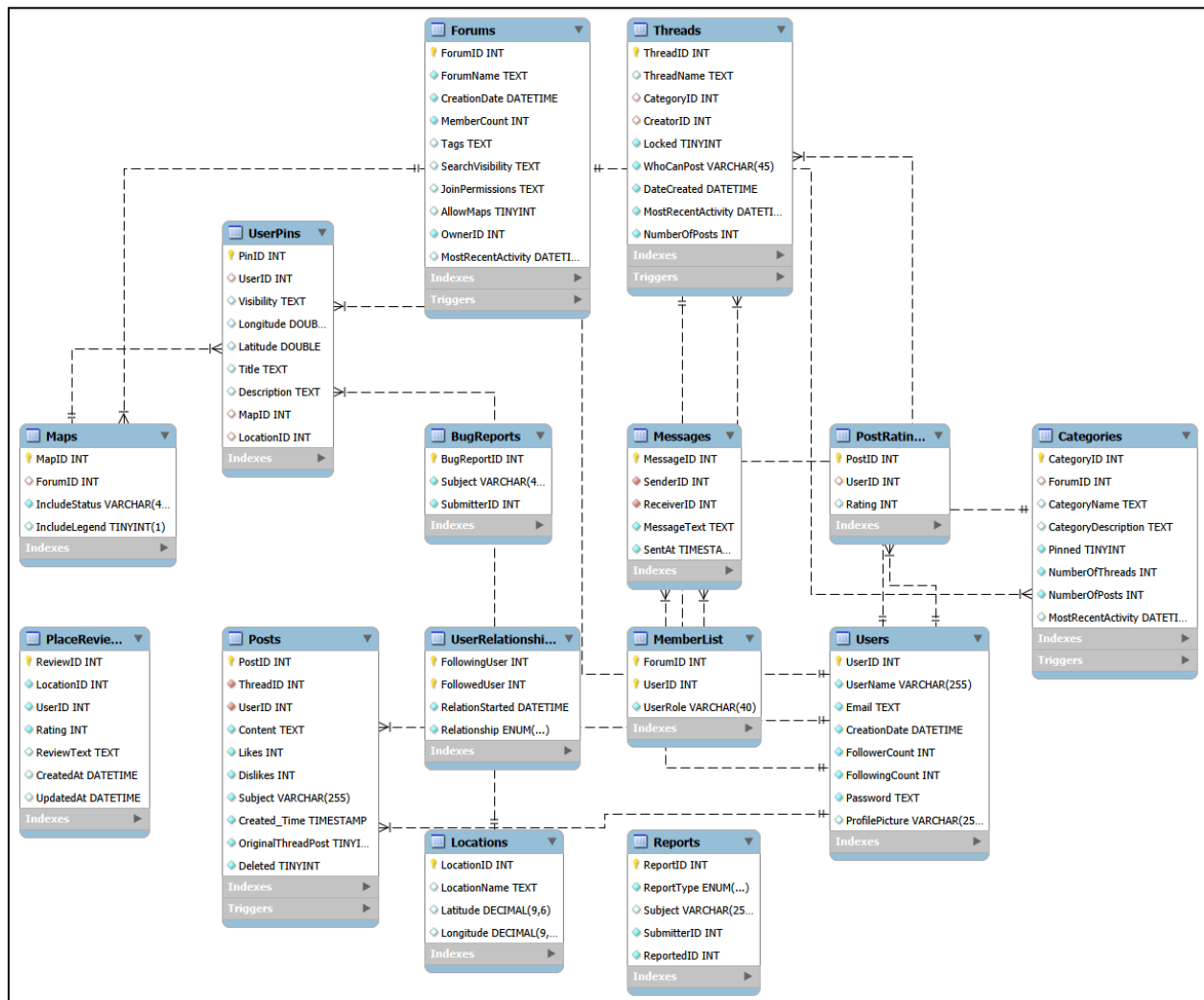


Figure 55: ERD

Sample Code

Chase Brown

Data loading functions:

The `getMapForForum` function as shown in figure 56 looks up the map tied to the forum the user selected by using `forumData.forumID`. It fetches the data from the `/map/forum/:forumID` endpoint and stores the returned `MapID` in the `currentMapID` variable.

```
const getMapForForum = () => {  
  if (!forumData?.forumID) return;  
  
  fetch(`http://localhost:5000/maps/forum/${forumData.forumID}`)  
    .then((response) => response.json())  
    .then((mapData) => {  
      console.log("map from backend:", mapData);  
  
      if (mapData && mapData.MapID) {  
        setCurrentMapID(mapData.MapID);  
      }  
    })  
    .catch((error) => console.error(error));  
};
```

Figure 56: *getMapForForum* function

The `loadPins` function as shown in figure 57 loads all of the user created pins for the current map from the `/userpins/map/:mapID` endpoint. It updates the `userPins` state so the markers can be rendered on the map.

```
const loadPins = (mapID) => {  
  if (!mapID) return;  
  
  fetch(`http://localhost:5000/userpins/map/${mapID}`)  
    .then((response) => response.json())  
    .then((pinList) => {  
      console.log("pins from backend:", pinList);  
      setUserPins(Array.isArray(pinList) ? pinList : []);  
    })  
    .catch((error) => console.error(error));  
};
```

Figure 57: *loadPins* function

The `loadReviews` function shown in figure 58 loads the user review data for a selected user created pin. This function makes two fetch calls, one fetches the `locationID` from `http://localhost:5000/place-reviews/locationID` while the other fetches the review summary (or average rating) from `http://localhost:5000/place-reviews/locationID/summary` and stores those values in `reviews` and `summary`.

```
const loadReviews = (locationID) => {
  fetch(`http://localhost:5000/place-reviews/${locationID}`)
    .then((res) => res.json())
    .then((data) => setReviews(data))
    .catch((error) => console.error(error));

  fetch(`http://localhost:5000/place-reviews/${locationID}/summary`)
    .then((res) => res.json())
    .then((data) => setSummary(data))
    .catch((error) => console.error(error));
};
```

Figure 58: *loadReviews* function

Pin related functions

The `handleRightClick` function shown in figure 59 runs when the user right clicks anywhere on the map they are viewing. When the user rightclicks, `setMenuPosition` saves the mouse position and displays the popup menu to create a pin while `setClickedLatLng` stores the longitude and latitude values for the pin placement.

```
const handleRightClick = (event) => {
  event.domEvent.preventDefault();

  setMenuPosition({
    x: event.domEvent.clientX,
    y: event.domEvent.clientY,
  });

  setClickedLatLng({
    lat: event.latLng.lat(),
    lng: event.latLng.lng(),
  });

  setSelectedPin(null);
  setSelectedRoadStatus(null);
};
```

Figure 59: *handleRightClick(event)* function

The `handlePlacePin` function as shown in figure 60 creates pins. The function first checks if the user is logged in, it returns the latitude and longitude data, confirms the map the user is viewing has a valid `mapID`, and confirms the users pin includes at least a title before allowing the user to create their pin. Once all of that information is confirmed the function then sets the values for

userID, mapID, visibility setting, longitude and latitude, title, and description and sends it to the database by sending a POST request to <http://localhost:5000/userpins>.

```
const handlePlacePin = () => {
  if (!loggedIn()) {
    alert("No logged in user found");
    return;
  }

  if (!clickedLatLng) return;

  if (!currentMapID) {
    alert("No map found for this forum");
    return;
  }

  if (!pinTitle.trim()) {
    alert("Pin must have a title");
    return;
  }

  const newPin = {
    user_id: currentUserID,
    map_id: currentMapID,
    visibility: pinVisibility,
    longitude: clickedLatLng.lng,
    latitude: clickedLatLng.lat,
    title: pinTitle,
    description: pinDescription,
  };

  fetch("http://localhost:5000/userpins", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(newPin),
  })
    .then((response) => response.json())
    .then(() => {
      loadPins(currentMapID);
      setPinTitle("");
      setPinDescription("");
      setPinVisibility("Public");
      setMenuPosition(null);
      setClickedLatLng(null);
    })
    .catch((error) => console.error(error));
};
```

Figure 60: *handlePlacePin* function

The *handleDeletePin* function as shown in figure 61 handles the deletion of user created pins. It first checks the userID to confirm the user is logged in and that the logged in userID matches the

userID that created the pin. If the userID does not match, the pin will not be allowed to be deleted by this user.

```
const handleDeletePin = (pinID) => {
  if (!user) {
    alert("No logged in user found");
    return;
  }

  fetch(`http://localhost:5000/userpins/${pinID}`, {
    method: "DELETE",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ userID: currentUserID }),
  })
    .then(async (response) => {
      const data = await response.json();

      if (!response.ok) {
        alert(data.message || "Failed to delete pin");
        return;
      }

      loadPins(currentMapID);
      setSelectedPin(null);
    })
    .catch((error) => console.error(error));
};
```

Figure 61: handleDeletePin function

The handleDeletePin function as shown in figure 62 handles the deletion of user created pins. It first checks the userID to confirm the user is logged in and that the logged in userID matches the userID that created the pin. If the userID does not match, the pin will not be allowed to be deleted by this user.

```
const handleDeletePin = (pinID) => {  
  if (!user) {  
    alert("No logged in user found");  
    return;  
  }  
  
  fetch(`http://localhost:5000/userpins/${pinID}`, {  
    method: "DELETE",  
    headers: {  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify({ userID: currentUserID }),  
  })  
  .then(async (response) => {  
    const data = await response.json();  
  
    if (!response.ok) {  
      alert(data.message || "Failed to delete pin");  
      return;  
    }  
  
    loadPins(currentMapID);  
    setSelectedPin(null);  
  })  
  .catch((error) => console.error(error));  
};
```

Figure 62: *handleDeletePin* function

The *handleUpdatePin* function as shown in figure 63 handles saving changes to an existing user created pin. It begins by confirming the user is logged in and has a valid *userID* and that the *userID* of the logged in user matches the *userID* of the creator of the pin. After passing that check the function sends a PATCH request to `http://localhost:5000/userpins` to modify the selected *pinID*. This function gives the user the option to update the pin title, description, and visibility of their pin.

```
const handleUpdatePin = () => {
  if (!user) {
    alert("No logged in user found");
    return;
  }

  if (!selectedPin) return;

  fetch(`http://localhost:5000/userpins/${selectedPin.PinID}`, {
    method: "PATCH",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      userID: currentUserID,
      title: editTitle,
      description: editDescription,
      visibility: editVisibility,
    }),
  })
    .then(async (response) => {
      const data = await response.json();

      if (!response.ok) {
        alert(data.message || "Failed to update pin");
        return;
      }

      loadPins(currentMapID);
      setIsEditing(false);
      setSelectedPin(null);
    })
    .catch((error) => console.error(error));
};
```

Figure 63: *handleUpdatePin* function

The *handleEditClick* function as shown in figure 64 starts edit mode for a selected pin. The function copies the current pin values and places the values into the edit form.

```
const handleEditClick = (pin) => {
  setIsEditing(true);
  setEditTitle(pin.Title || "");
  setEditDescription(pin.Description || "");
  setEditVisibility(pin.Visibility || "Public");
};
```

Figure 64: *handleEditClick* function

Map interaction functions

The *handleMapclick* function as shown in figure 65 clears the temporary UI state when the map is clicked. This function closes the right-click menu and deselects the currently selected pin, business, or road status.

```
const handleMapClick = () => {  
  setMenuPosition(null);  
  setSelectedPin(null);  
  setSelectedRoadStatus(null);  
};
```

Figure 65: *handleMapClick* function

The `getRoadStyle` function returns the styling for each type of road status and assigns a color based on the road status whether the road is closed, under construction, or an incident has occurred such as a car accident.

```
const getRoadIcon = (type) => {  
  if (!window.google) return undefined;  
  
  let fillColor = "■ #f57c00";  
  
  if (type === "closed") {  
    fillColor = "■ #d32f2f";  
  } else if (type === "incident") {  
    fillColor = "■ #1976d2";  
  }  
  
  return {  
    path: window.google.maps.SymbolPath.CIRCLE,  
    scale: 8,  
    fillColor,  
    fillOpacity: 1,  
    strokeColor: "■ #ffffff",  
    strokeWeight: 2,  
  };  
};
```

Figure 66: *getRoadIcon* function

***useEffect* hooks**

The `useEffect` function shown in figure 67 automatically loads the road statuses for the currently selected map by fetching the road status data from the `http://localhost:5000/road-status` endpoint and displaying the status data to the correct map using the `forumID` as the identifier.

```
useEffect(() => {
  if (!forumData?.forumID) return;

  fetch(`http://localhost:5000/road-status/${forumData.forumID}`)
    .then((response) => response.json())
    .then((data) => {
      setRoadStatuses(data);
    })
    .catch((error) => console.error(error));
}, [forumData]);
```

Figure 67: Function to load road statuses

The useEffect function show in figure 68 confirms the mapID of the selected map within the selected forum. Once the mapID is known, the next useEffect function will run to display all of the existing user created pins.

```
useEffect(() => {
  getMapForForum();
}, [forumData]);

useEffect(() => {
  if (currentMapID) {
    loadPins(currentMapID);
  }
}, [currentMapID]);
```

Figure 68: Function to fetch mapID and load pins

Sean Kehoe

Post Editing

The post editing feature has two core components. The first is the editpost function which is responsible for sending the PATCH request to the backend that will amend the post's content field in the database with the newly entered data from the user while also appending a message that the post has been edited from its original state. This allows a user to issue a correction quickly and easily while also letting others know that the post was altered and may not totally reflect what previous responses were talking about. It works by directly accessing the post's

entry in our database via the route “http://localhost:5000/posts/:id” wherein ‘:id’ is substituted for the post’s ID via `$(props.PostID)` (as shown in line 3 of Figure 69). The second is the dialog modal that appears when the edit button is clicked which houses the data entry form. This is responsible for collecting the changes the user wishes to make to their post and calling the `editpost` function only once the submit button is clicked. (as shown in Figure 70) If the user decides during the editing process to not change their post then they can click the cancel button which will close the dialog modal and not send any changes to the database. The design choice behind using a modal is to enhance the user experience by creating minimal interruptions during their use of the site as it remains hidden until specifically called for by a user thus preventing cluttering of the screen with more options than would be necessary at any time. Furthermore, once either submit, cancel, or the escape key are clicked the modal will close automatically thus stopping it from staying on screen and preventing further use of the site. Lastly, once the decision to submit is made the page is forced to reload so that a user is able to instantly and clearly see that their post has been updated and is displaying properly. (as shown in Figure 71)

```
const editpost = async () => {
  try { // submit to posts table to update data
    const response = await fetch(`http://localhost:5000/posts/${props.PostID}`, {
      method: "PATCH",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({Content: text + "    This post has been edited."}),
    });

    if (!response.ok) {
      throw new Error("Network response error");
    }
  } catch (error) {
    console.log(`Data was submitted unsuccessfully: ${error}`);
  }
  window.location.reload(); // reload window to show that profile has been updated.
}
```

Figure 69: Code for post edit

```
{ sampleUserID == props.UserID &&
  <div className="delete-edit">
    <button className="delete-button" onClick={deletepost}>Delete</button>
    <button className="edit-button" command="show-modal" commandfor={modalID}>Edit</button>
  </div>
  <dialog id={modalID} className="edit-field" >
    <div className="edit-container"><input
      className="text-field"
      value={text}
      onChange={(e) => setText(e.target.value)}
      placeholder={props.Content}
      name= "ContentEdit"
    />
    <button onClick={editpost} className="submit">
      Submit
    </button>
    <button className="Close" command="close" commandfor={modalID} value="cancel">
      Close
    </button></div>
  </dialog>
</>
}
```

Figure 70: HTML for post edit

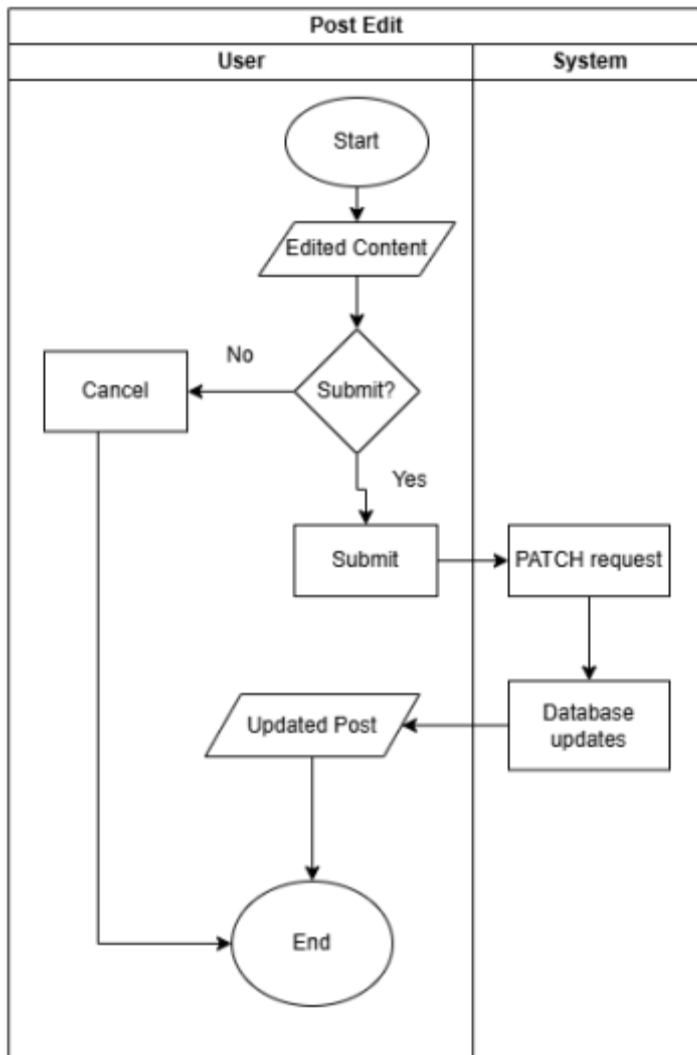


Figure 71: Flowchart diagram for post edit

Post Creation

The post creation utilizes a modular form that was created for the site that can be used across a wide variety of use cases (e.g. update profile, post creation, etc.). Through this form we are able to specify an endpoint based upon the passed in props, shown in code as `props.endPoint` which will direct to `http://localhost5000/posts`. This alongside two other variables, `props.method` which sends as POST in this instance and `processedData` which in this case will be the post's content, the creator's `userID` and `username`. (shown in figure 72) The data for post creation is

first gathered from a mixture of user input and preset data (e.g. method and endpoint) into the text section, which is the only element for the post creation form, and then it is stored alongside the user's ID and username in the `processedData` variable as part of a key:value pair. The data stored in the `processedData` variable is then used in a `universalDatabaseInteraction` call to store the data in the posts table in the database. The `console.log` function is used for testing purposes to determine if the data is being captured properly. The form itself is rendered at the bottom of any forum thread that a user is permitted to post in. (as shown previously in figure 22). A user simply types their message/question into the text field of the form and clicks the create post button to submit their post.

```
const handleSubmit = async (event) => {
  event.preventDefault();

  let processedData = {};

  new FormData(event.currentTarget).forEach((value, key) => {
    processedData[key] = value;
  });

  for (let i = 0; i < props.passToEndPoint.length; i++) {
    let keyPair = props.passToEndPoint[i];
    processedData[keyPair.key] = keyPair.value;
  }

  console.log(props.endPoint);
  console.log(processedData);

  universalDatabaseInteraction(props.method, props.endPoint, processedData).then((data) => {
    console.log(data);

    if (props.updateUserInformation) { // reset user information and go to profile
      setUser({ UserID: parseInt(processedData.UserID), Username: processedData.Username });

      navigate(`/Profile/${processedData.Username}`);
    }
    window.location.reload(""); // reload window to show data change
  });
}
```

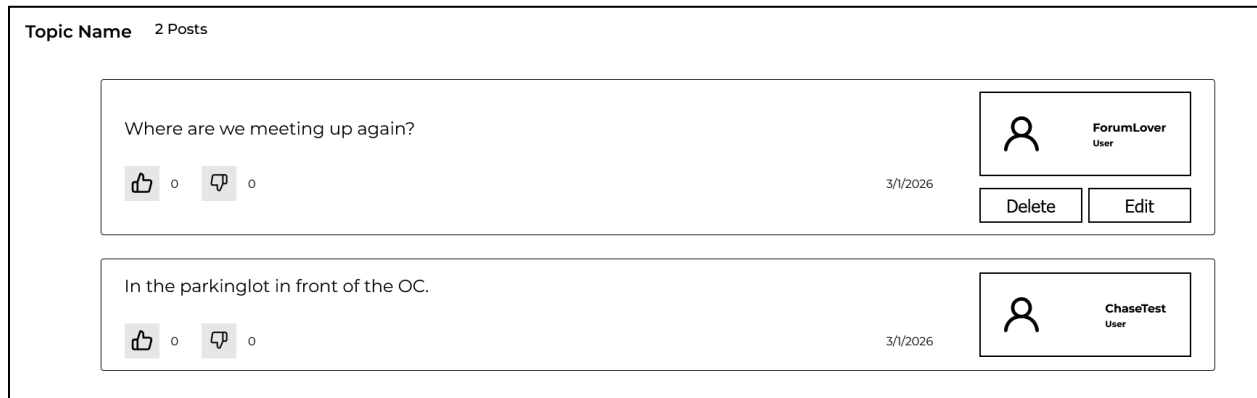
Figure 72: Code for Form Element

```
function FormTextSection( props ) {  
  const [sectionInputs, setSectionInputs] = useState(null);  
  // props = sectionTitle=text type=text options: {[id value text checked (if type is radio)]}  
  
  return (  
    <>  
      <div className="form-section form-text-section">  
        <div className="section-heading">  
          <h2 className="forum-section-header">{ props.sectionTitle }</h2>  
        </div>  
        <div className="section-options">  
          <input type="text" id={ props.sectionID } name={ props.sectionID }/>  
        </div>  
      </div>  
    </>  
  )  
}
```

Figure 73: Code for Form Text Section

```
return (  
  <>  
    <div className="form-component">  
      <h1 className="form-title"> { props.formTitle } </h1>  
      <form onSubmit={ handleSubmit } action="">  
        {props.children}  
      </form>  
    </div>  
  </>  
)
```

Figure 73a: Code for Form HTML

Rory Strachan***Changing User Roles Within A Forum***

Upon entering a forum landing page, if the user is a forum administrator or forum owner, they will be able to access the forum administrator dashboard. This dashboard can be viewed on the right of figure 74 The dashboard is organized into numerous sections, all settings pertaining to forum users are stored in “Member Settings”, which can be viewed in figure 75.

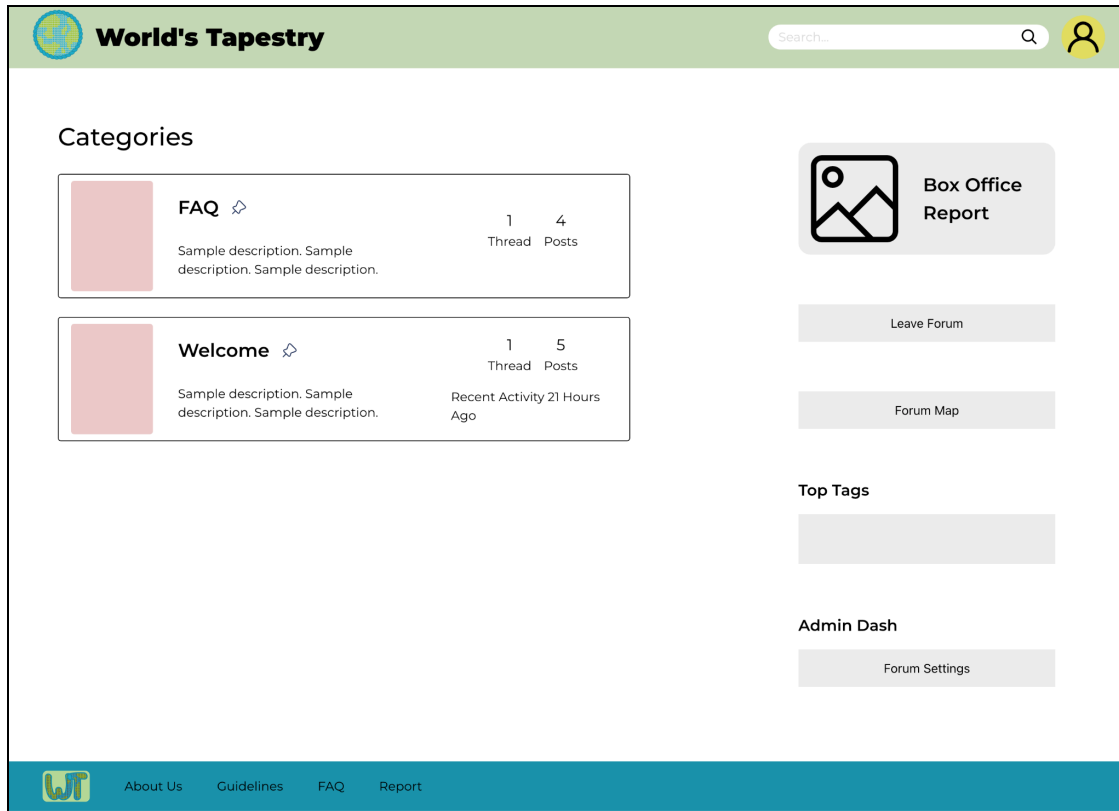


Figure 74 Forum Landing Page, The Admin Dash is visible on the bottom of the right sidebar

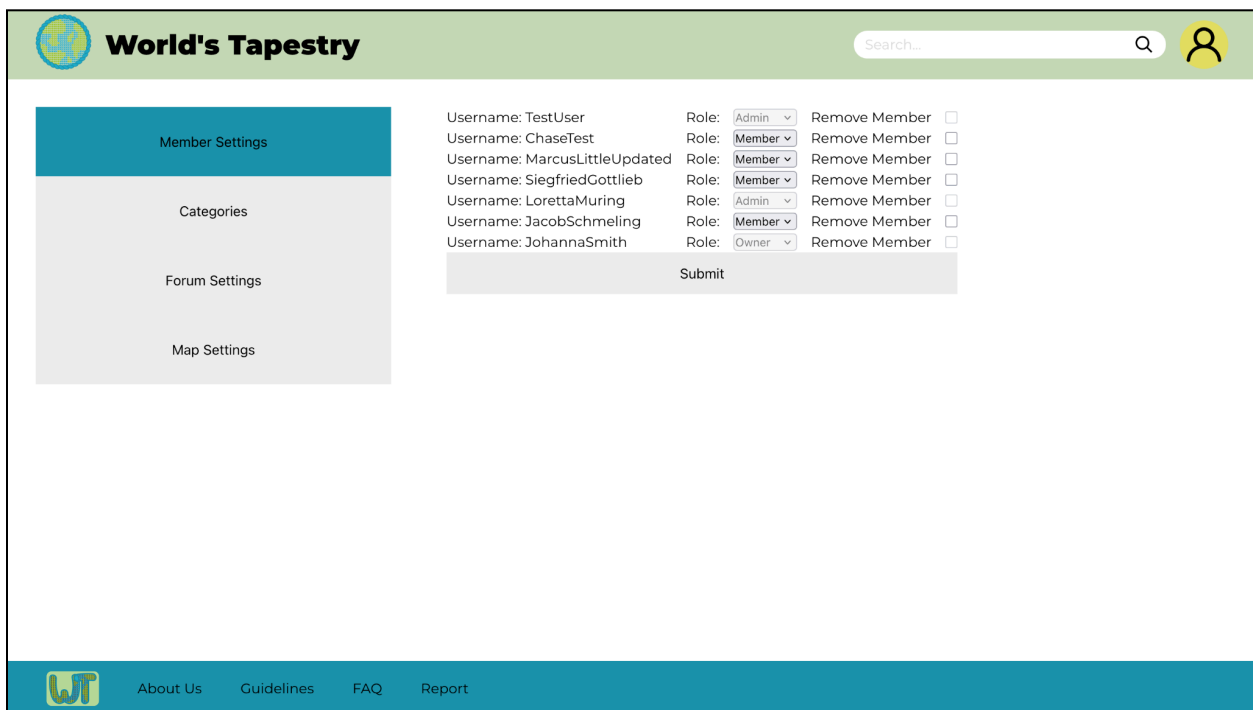


Figure 75 Admin Dash member settings section, the UI is not final.

The section for changing user roles overlaps with removing members, code snippets will include details about user removal, but focus will be placed on changing the role of forum members.

The role change setting is one large form component, “MemberListForm”, that holds numerous small “FormRow” components. This can be viewed in figure 76.

```
<>
<div className="user-forms">
  <form onSubmit={ processForm } action="">
    {forumMembers.map((member) => {
      return <FormRow key={member.UserID} forumMember={member} adminUserRole={userRole} ForumID={forumID}/>
    })}
    <button id="forum-member-edit-submit">Submit</button>
  </form>
</div>
</>
```

Figure 76: HTML output of MemberListForum

When the user loads into the admin dashboard, the program will fetch all the users that have joined the forum. As shown in figure 77, this is a simple get request for all rows in the MemberList table. MemberList stores the relationships between users and forums as well as the roles users hold within forums. The SQL query, as viewed in figure 78 joins the MemberList table to the User table so the following functions are able to process with as much information about the users of the forum as possible.

```
const getForumMembers = () => {
  fetch(`http://localhost:5000/${forumID}/users`)
    .then(response => response.json())
    .then(memberList => {
      setForumMembers(memberList);
    }).catch(error => console.error(error));
}
```

Figure 77: Fetch request for all the users in the forum.

```
// GET users who are in a certain forum
app.get("/:forumID/users", (req, res) => {
  const { forumID } = req.params;

  const SQL =
    `SELECT DISTINCT * FROM MemberList
     INNER JOIN Users u ON MemberList.UserID = u.UserID
     WHERE MemberList.ForumID = ?`;

  db.query(SQL, [forumID], (err, results) => {
    if (err) return res.status(500).json(err);
    res.json(results);
  });
});
```

Figure 78: SQL for getting users who are in a specific forum.

The array of users is saved to a state. This is what the function `setForumMembers` is doing. It is best to store data that is relevant to react components in a react state so data can be passed between states in an optimal and clean manner. `MemberListForm` uses a map to render a `FormRow` for each user that has joined the form (as shown in figure 79). `FormRow` takes data about the forum member and data about the administrator using the dashboard feature as props.

Each row in the forum is dynamically generated. As visible in figure 79, the drop down menu on the form corresponding to a user's role will default to their current role. The forum administrator who is accessing the dashboard will only be allowed to remove users or change the roles of users who hold a lower level of access. This means that administrators cannot change the role of other administrators or forum owners. Forum owners are the only users that can remove administrators and change the roles of administrators. As shown in figure 79, options in a row will be disabled if the admin does not have access to change the users role

```

<>
<div className={`form-list-element admin-can-change-${privilegeOverUser}`}>
  <div className="user-form-section">
    <p>Username: {userName}</p>
  </div>
  <div className="user-form-section">
    <label htmlFor="user-form-name-role">Role:</label>
    <select disabled={!privilegeOverUser} defaultValue={ userRole } id={` ${userID}-change-role` } name={` ${userID}-change-role`}>
      <option value="Member">Member</option>
      <option value="Admin">Admin</option>
      { userRole == "Owner" &&
        <option value="Owner">Owner</option>
      }
    </select>
  </div>
  <div className="user-form-section">
    <label htmlFor="remove-member">Remove Member</label>
    <input disabled={!privilegeOverUser} type="checkbox" id={` ${userID}-user-remove` } name={` ${userID}-user-remove` } value="1"/>
  </div>
</div>
</>

```

Figure 78: Code for generating FormRows

After the admin has changed the data they have wanted to, they will press submit. This will send the data from all the forum rows to a chain of functions, starting with processForm shown in figure 79, that will process the data (figure 79), and then send the data to the database (figure 79).

```

const processForm = async (event) => {
  event.preventDefault();

  const processedFormData = processFormData(new FormData(event.currentTarget));

  for (let key in processedFormData) {
    sendData(key, processedFormData[key].value, processedFormData[key].type);
  }
}

```

Figure 80: On form submit the data is sent to be processed, and then sent to the database

```

const processFormData = (formData) => {
  let processedData = {};

  formData.forEach((value, key) => {
    const userID = key.split('-')[0]; // from how the object is set up everything before the first hyphen will be the user id
    const valueToChangeTo = value;

    const type = key.split('-')[2]; // will either be role or remove

    if (userID in processedData) {
      if (processedData[userID].type === "role") { // delete takes precedence over a role change
        processedData[userID] = {type: type, value: valueToChangeTo};
      }
    } else {
      processedData[userID] = {type: type, value: valueToChangeTo};
    }
  });

  return processedData;
}

```

Figure 81 Data processing function

```

const sendData = (userID, value, type) => {
  if (type === "remove") {
    fetch(`http://localhost:5000/remove-user-from-forum`, {
      method: "DELETE",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({UserID: userID, ForumID: forumID})
    }).then(response => {
      window.location.reload();
    }).catch(error => console.error(error));
  } else if (type === "role") {
    fetch(`http://localhost:5000/memberlist-change-role`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({UserID: userID, ForumID: forumID, UserRole: value})
    }).then(response => {
      window.location.reload();
    }).catch(error => console.error(error));
  }
}

```

Figure 82 Data is sent to the database

The data is processed by determining if forum members' role is changing or if they are being removed from the forum. If a member is marked for a role change and a removal, removal will take precedence and the role change will be ignored. Data from the form is processed into an object where keys corresponding to UserIDs and the values will be an array that contain the type,

removal or role change, and the value of the key, 1 for removal or “Member” or “Admin” for a role change.

This data object is returned to the processForm function, which then loops over all the keys in the object and sends each key value pair to the database depending on if it is a role change or removal.

For role changing, a put request is made to the backend, as shown in figure 82, The SQL query can be viewed in figure 83. The MemberList uses UserID and ForumID as a primary key, so we simply use the forum member's id and the forum's id to locate the corresponding MemberList row and replace the user's role with the new value.

```
app.put("/memberlist-change-role", (req, res) => {
  const { UserID, ForumID, UserRole } = req.body;

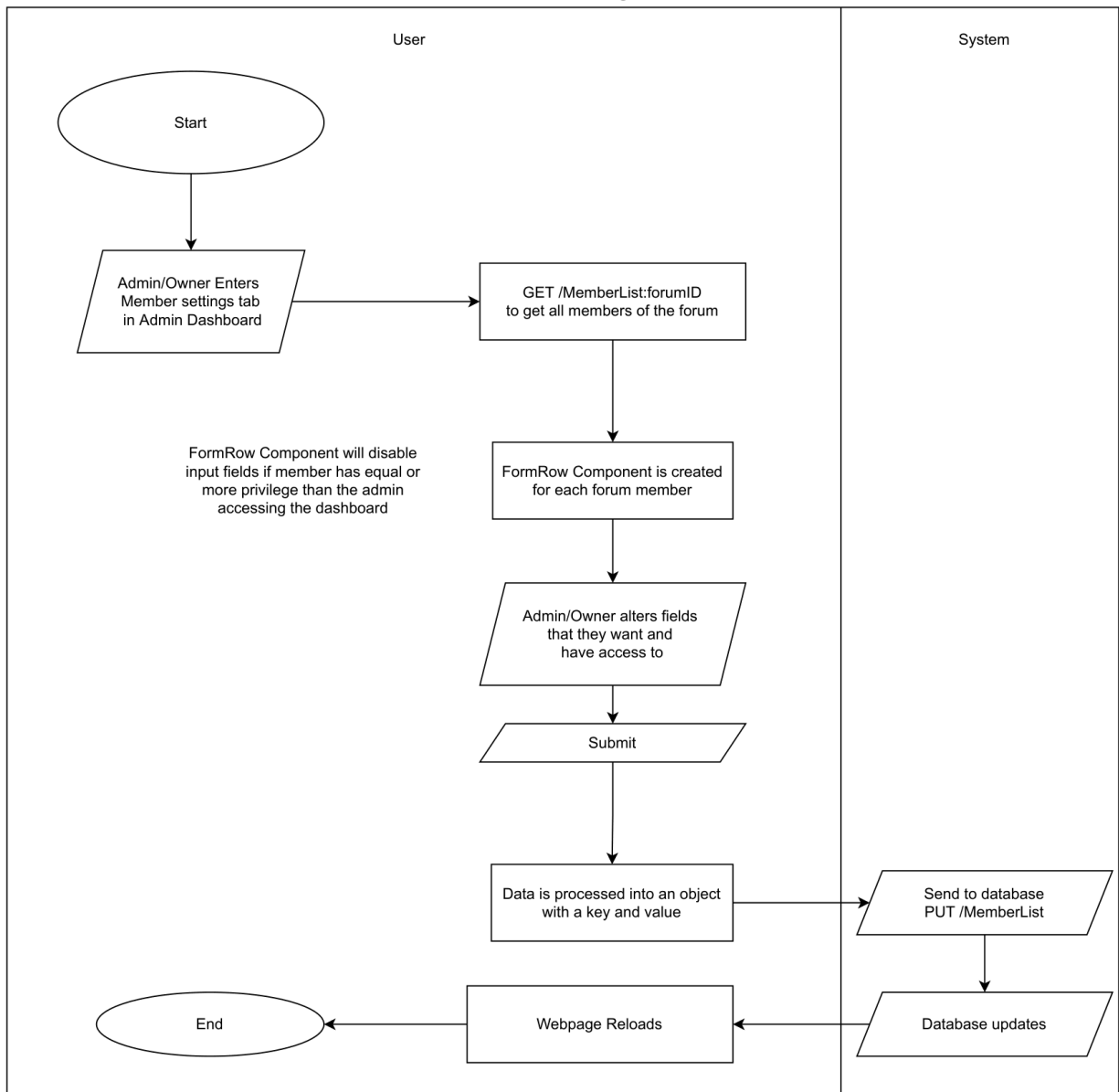
  const sql = `
    UPDATE MemberList
    SET UserRole = ?
    WHERE UserID = ? AND ForumID = ?
  `;

  db.query(sql, [UserRole, UserID, ForumID], (err, result) => {
    if (err) {
      console.error(err);
      return res.status(500).json({ error: "Failed to update post" });
    }
    res.json({ message: "Post updated" });
  });
});
```

Figure 83 SQL query for role changing

The previously described algorithm is illustrated in figure 67n.

Forum Role Editing



UserID	UserName	Email	CreationDate	FollowerCount	FollowingCount	Password	ProfilePicture
1	forumlover	user@gmail.com	2026-03-25 03:58:59	0	0	1	avatar3.png
2	berries500	berry@gmail.com	2026-03-25 03:59:18	0	0	1	avatar4.png
3	xx_xx	tshirt@gmail.com	2026-03-25 04:01:36	0	0	1	avatar1.png
4	700flowers	berry2@gmail.com	2026-03-25 04:01:36	0	0	1	avatar2.png
5	metalmario	metal@gmail.com	2026-03-25 04:01:36	0	0	1	avatar10.png
6	ILOVECOMPUTER	pc@gmail.com	2026-03-25 04:01:36	0	0	1	avatar6.png
7	IHATECOMPUTER	nopc@gmail.com	2026-03-25 04:01:36	0	0	1	avatar8.png
8	playstation7	ps7@gmail.com	2026-03-25 04:01:36	0	0	1	avatar5.png
9	segasaturn	saturn@gmail.com	2026-03-25 04:01:36	0	0	1	avatar7.png
10	laurence	laurence@gmail.com	2026-03-25 04:01:36	0	0	1	avatar9.png
11	MESSAGEFORFR...	scam@gmail.com	2026-03-25 04:01:36	0	0	1	avatar7.png

*Figure 84 Role editing flowchart****Editing Forum Tags***

Upon creating a forum, the creator will be allowed to enter in a comma separated list of tags. The data is cleaned, Figure 85, so this tag list will have no duplicates, no double quotes, single quotes, backticks, or comms, and the tag itself cannot be an empty string or a string of only spaces. This data is stored in the backend as a string. Given more time this may be improved to be its own table.

```
const cleanTags = (tags) => {  
  let tagArray = tags.split(",");  
  let tagSet = new Set(); // prevents duplicates  
  
  for (const tag of tagArray) {  
    let trimmedTag = tag.trim();  
  
    if (trimmedTag !== "" && !tag.includes("\"") && !tag.includes("'") && !tag.includes("`") && !tag.includes(",")) {  
      tagSet.add(trimmedTag);  
    }  
  }  
  
  return Array.from(tagSet).join(",");  
}
```

Figure 85: Function for cleaning tag data

Editing tags uses many of the same tools used to set the tags up. The interface is located in the Admin Panel under the “Community Settings” tab, as shown in figure 86. Only admins and forum owners will be allowed to update tags. The tag data is stored as a prop in this component, all forum will be fetched when the user enters the forum.

Figure 86: Updating tags interface.

This form uses a custom form component, figure 87, that takes in an end point and method so the data submission process can be as simple as possible. The custom form component is used as the parent, a textarea is nested as its child, as shown in figure 88. The current tags are set as the default value for the text area.

```
const handleSubmit = async (event) => {
  event.preventDefault();

  let processedData = {};

  new FormData(event.currentTarget).forEach((value, key) => {
    processedData[key] = value;
  });

  for (let i = 0; i < props.passToEndPoint.length; i++) {
    let keyPair = props.passToEndPoint[i];
    processedData[keyPair.key] = keyPair.value;
  }

  universalDatabaseInteraction(props.method, props.endPoint, processedData).then((data) => {
    window.location.reload(""); // reload window to show data change
  });
}
```

Figure 87: Custom React component

```

<div className="form-textarea">
  <FormElement formTitle="Update Tags" endPoint="update-tags" method="PUT" passToEndPoint={ [{key: "ForumID", value: forumID}] }>
    <textarea name="ForumTags" id="ForumTags" defaultValue={forumTags}></textarea>
    <button type="submit" className="change-forum-settings">Update</button>
  </FormElement>
</div>

```

Figure 88: React components for the tag updating forum.

If given more time, this system would be updated to maybe feature buttons and a separate space for entering in new tags, at this time this system is simple and works best. The user simply has to update the field and press the update button to submit the new tags.

On form submit a put request is made to the endpoint “update-tags”, as shown in previous figure 89. Once sent to the backend, the backend will clean the tags in the same manner as done when creating a forum. Then an update request is made to SQL, the Forum row matching the provided ForumID will have its tags replaced with the new content.

```

app.put("/update-tags", (req, res) => {
  const { ForumID, ForumTags } = req.body;

  let tags = cleanTags(ForumTags);

  const sql = `
    UPDATE Forums
    SET Tags = "${tags}"
    WHERE ForumID = ${ForumID}
  `;

  db.query(sql, (err, result) => {
    if (err) {
      console.error(err);
      return res.status(500).json({ error: "Failed to update tags" });
    }

    res.json({ message: "tags updated" });
  });
});

```

Figure 89: SQL update request for tags

ForumID	ForumName	CreationDate	MemberCount	Tags	SearchVisibility	JoinPermissions	AllowMaps	Owner
1	Book Club	2026-03-25 04:02:03	1	Books, Learning, Reading, writing, stories	Searchable	Anyone	1	1
2	Game club	2026-03-25 04:02:32	1	Games, Media, Club, Social	Searchable	Anyone	0	8
16	Rochester Biking	2026-03-25 04:06:40	1	Michigan, Rochester, Biking, Active	Searchable	Anyone	1	5
17	Movie Review Hub	2026-03-25 04:09:07	1	Movies, Media, Entertainment, Cinema	Searchable	Anyone	1	2

Figure 90: Forum table, zoomed in to focus on the tags column

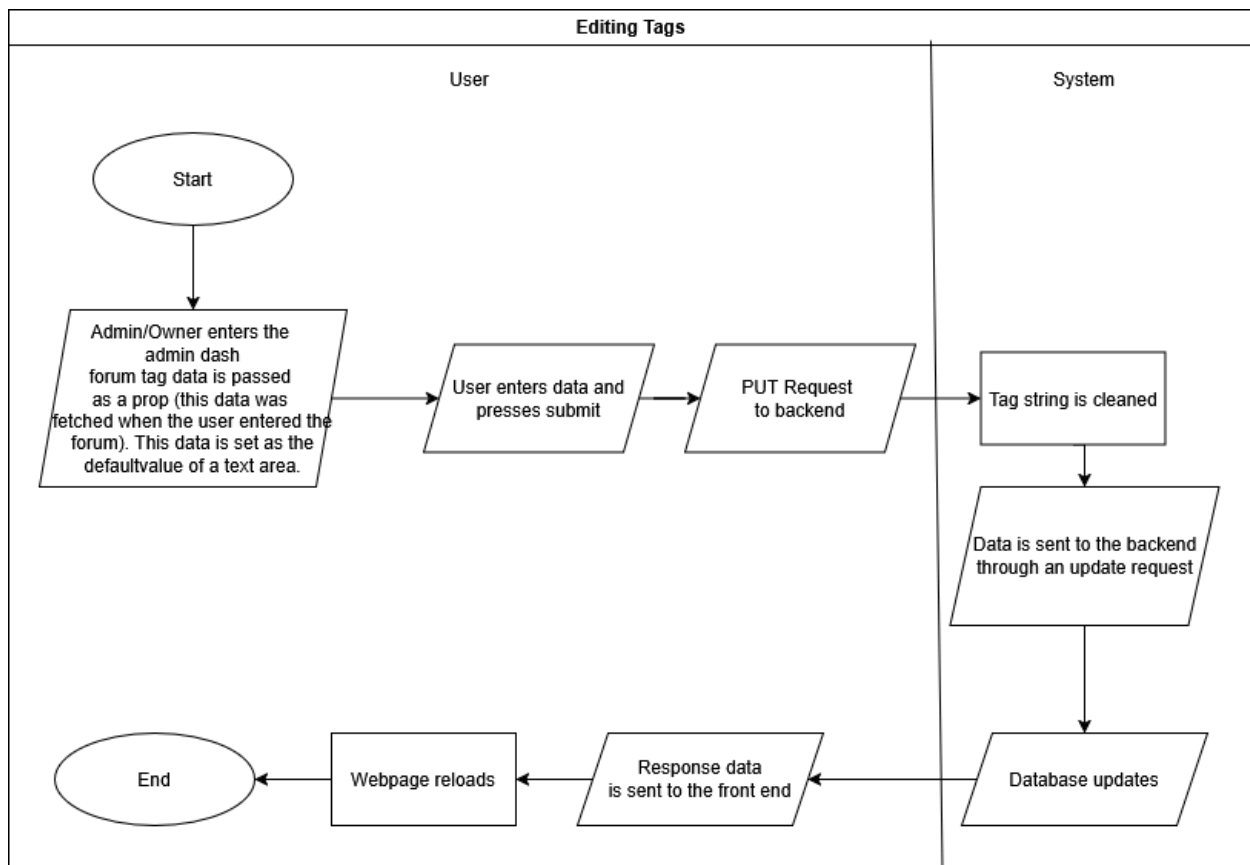


Figure 91: Tag editing flow chart

Jennifer Orlando

About Us Page

What follows is a sample of how pages are linked together in react. All pages are routed through the React function BrowserRouter. The BrowserRouter grabs the function that is the page the user intends to visit. React then renders the page for the user to interact with.

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <>
      <BrowserRouter>
        <Routes>
          <Route path="/AboutUs" element=<Layout />>
            <Route index element=<AboutUsPage />/>
          </Route>
        </Routes>
      </BrowserRouter>
    </>
  )
}

import { useState, useEffect } from "react";
import './FooterLinkPages.css';

const AboutUsPage = () => {

  return (
    <>
      <div className="main-content">
        <h1>Hello!</h1>
        <p>We are a group of Oakland University students<br><br><br><br>
          • Rory Strachan<br>
          • Chase Brown<br>
          • Sean Kehoe<br>
          • Mohammed Qudaih<br>
          • Jennifer Orlando<br>
        </p>
      </div>
    </>
  )
}

export default AboutUsPage;
```

Figure 92 Code for page rendering

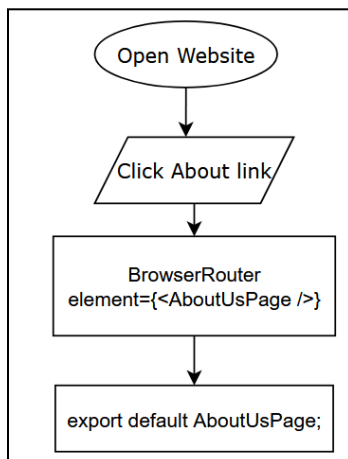


Figure 93 Flowchart for page rendering

Report Feature

There are buttons on the community, thread, and posts that all lead to form with a text box for the user to fill out any issue they're having.

Figure 94: UI for report feature.

The form sends to the database the information that leads the user to the report form. Where did the user come from- community, thread, or post- with the id that will show the content that is an issue. The user who submits the issue has their id stored in the database.

```
<div className="side-bar-section">
  <Link key={` ${forumID}-${forumName}-report` } className="router-link"
    to={ `/Report-Content` }
    state={{
      reportedID: forumID,
      reportedName: forumName,
      type: "Forum"
    }}
  >
    <button className="forum-settings" > Report Forum </button>
  </Link>
```

Figure 95: Code for linking to report form.

```

export const ReportForm = (props) => {
  const state = useLocation().state;

  const { user, setUser } = useContext(UserContext);
  const [type] = useState(state.type);
  const [reportedID] = useState(state.reportedID);
  const [reportedName] = useState(state.reportedName);

  console.log(state)

  let issueField = { };

  const navigate = useNavigate();

  return (
    <>
      <div className="form-component content-report-form">
        <FormElement formTitle={`Report ${type} ${reportedName}`} endPoint="content-reports" method="POST"
          passToEndPoint={ [{key: "SubmitterID", value: user.UserID},
            {key:"ReportedID", value: reportedID},
            {key:"ReportType", value: type} ] }>
          <FormTextSection type="text" sectionTitle="Describe your issue" sectionID="Subject"/>
          <button type="submit" className="report-form">Submit</button>
        </FormElement>
      </div>
    </>
  )
}

```

Figure 96: Code for report form.

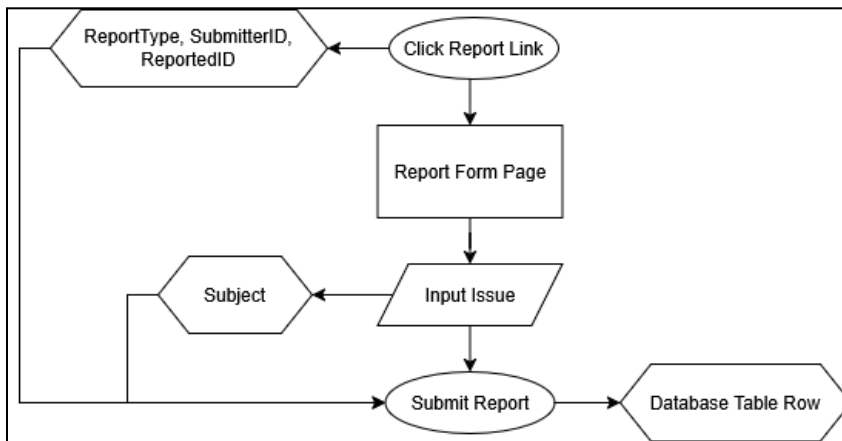


Figure 97 Flowchart for reporting

Mohammed Qudaih***User Profile Viewing***

This code is responsible for retrieving and displaying a user's profile information. When the profile page loads, the frontend sends a GET request to the backend API using the route `/api/users/:id`. The ID parameter is used to identify which user's data should be fetched.

On the backend, the Express server receives the request, extracts the user ID from the parameters, and executes a SQL query on the Users table to find the matching record. If the user exists, their information is returned as a JSON response. If no user is found, an error response is sent instead.

Once the frontend receives the response, the data is stored in state and rendered on the profile page. This allows the application to dynamically display user details such as username, email, role, and account creation date.

This implementation demonstrates the full data flow between the frontend, backend, and database. One challenge encountered was handling invalid or missing user IDs, which was resolved by adding proper error handling in the backend.

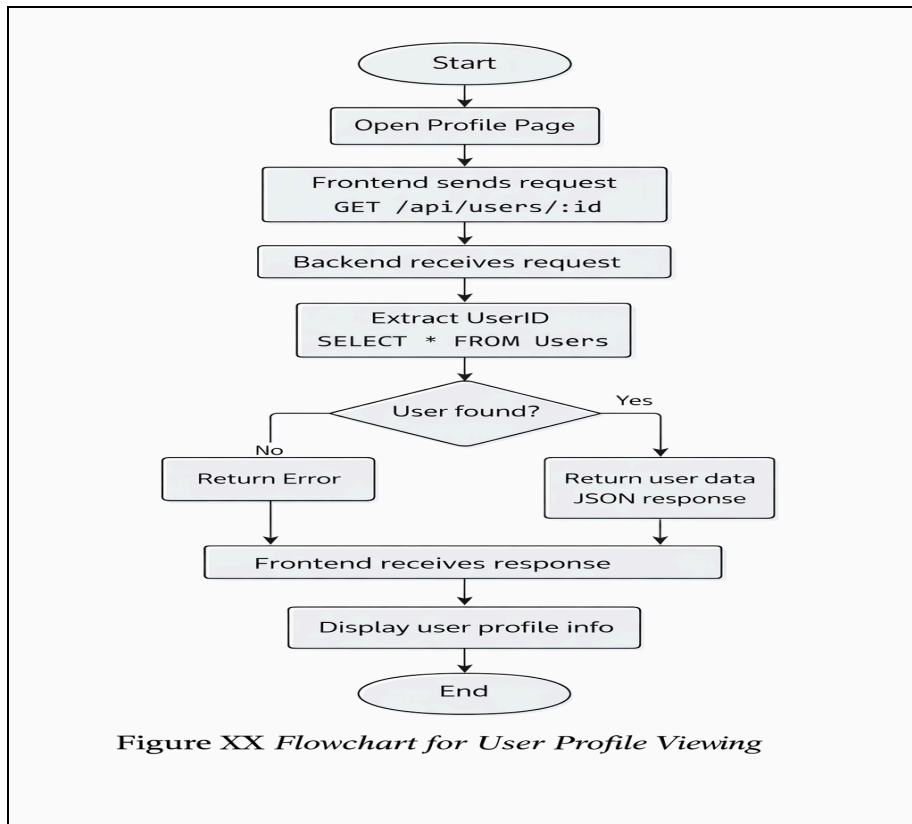
This code review focuses specifically on how the backend API processes requests and how data flows between the frontend, backend, and database.

```
// Fetch user profile data from backend
useEffect() => {
  fetch("http://localhost:5050/api/users/${userId}")
    .then((res) => res.json())
    .then((data) => setUser(data))
    .catch((err) => console.error("profile fetch error:", err));
}, [userId];
}
```

Figure 98

```
// Backend route to retrieve a specific user profile
app.get("/api/users/:id", (req, res) => {
  const id = req.params.id;
  db.query(
    "SELECT * FROM Users WHERE UserID = ?",
    [id], (err, results) => {
      if (err) return res.status(500).json(
        {error: "Query failed"})
      if (results.length === 0) (error: "User not found"));
      res.json(results[0]),
    }
  );
});
```

Figure 99

*Figure 100*

ADDIE Model Discussion

Analysis

The first step in the analysis process was to first identify a current underserved need in the world in the case of this project that would be the ability for people to come together freely and form communities of their own. Next was to address the best possible way to achieve that goal with potential services such as social media applications, messaging applications, or forum sites being the stand out candidates. From there it was decided that the most feasible and best option would be a forum site as they are simpler in structure, allow free communication between individuals with the ability for moderation and administration, and would take the least amount of time to develop effectively. This then led into the question of what subsystems would need to

be developed in order to effect the site's goal of bringing people together with the winning candidates being: user profile tools and direct messaging, forum interaction, forum admin tools, moderation, and a map for localized communities. These were chosen after looking towards similar sites and applications like Reddit, Discord and Giant in the Playground, and identifying what their common and discernible features and subsystems were and to then decide what World's Tapestry's distinguishing feature would be while maintaining a streamlined user experience.

Design

After conducting the analysis phase and creating a rough estimate of the necessary subsystems it became necessary to design the site's structure and each of the subsystems' various features. This led into the decision of the first of three guiding principles: that the site must keep users in control of their experience. This was emphasized heavily throughout the site as it forms the basis of its design philosophy as the user is ultimately in control of their experience on the site such as what communities they wish to join, what threads to post in, and with whom they wish to communicate. Furthermore, it was deemed a priority for the site's UI to be easy to navigate and uniform so as to avoid users being driven away due to frustration. A prime example of this would be the decision to create a customizable, modular form that could be used for a variety of features that would allow a user to recognize when they will be making an impactful action (i.e. updating profile data, creating a post, etc.). From there it was necessary to ensure that users would be able to correct their mistakes as any miscommunication online could result in fracturing a relatively tenuous community or relationship. To best facilitate these guiding principles multiple wireframes were created to create a visual representation of the site and how it would reflect them.

Development

The workload for the development process was split amongst all members of the project with each member taking a subsystem to develop. It was noticed early on that some subsystems, forum interaction and admin tools, had a one-way heavy dependence (e.g. interaction needed admin tools' forum creation feature first) and could not exist until the feature was created so admin tools was given the highest priority in the development cycle and would be developed first. After this initial stage it was found that the various subsystems could be developed individually to avoid the project falling behind schedule in the event one of the group members became incapacitated or otherwise unable to work.

Implementation

Implementation was continuous over the course of the project as a live model of the site was necessary for testing and UI refinement. The best deemed course of action for the site's implementation was to host it locally and keep the database on a secured Amazon web server so that all members would have an up-to-date, uniform and easily accessible instance thus reducing the chances someone would be operating on out-of-date data.

Evaluation

Each step of the project saw constant evaluation from the group. For instance, during the initial analysis phase each member weighed in on the initial findings and offered critical feedback that would lead into design decisions. During the design phase the site was constantly re-evaluated both in terms of what features were feasible to develop, and overall design and UI. Then during implementation the primary concerns for evaluation were again the site's design but also security concerns for both users and the site itself.

Future Updates

Map

One possible improvement to the map subsystem would be to implement the ability for users to map out routes. This would make it easier for users to visit other users points of interest that they have highlighted with a pin, increasing community engagement.

User Personalization

To further increase user engagement an improvement we could add is the ability for users to customize more features. Like different color profiles for communities and user profile pages. Having users be able to upload more pictures. Those pictures could be used for albums and backgrounds.

Safety Features

Currently there are instances where private information is displayed, such as email addresses. We would like to add an option for users to decide what information about them to hide or display on their profile. Passwords are typed in without a feature to hide the characters inputted.

Events

Events are an important feature of communities, and as such we feel it is important to eventually include them on our site. To have the ability for a community admin to add events that users in the community can view details about the event and respond with their intention about it.

Appendices

Appendix A: System Vision Document

Problem Description

As the world becomes smaller through technology and constant connectivity, local communities often feel increasingly disconnected. Local information such as business updates, road closures, and local news is now spread across multiple platforms and can be difficult to access in real time.

At the same time, residents often lack a dedicated space to engage with their neighbors to share concerns, have constructive discussions, and build a relationship with those nearby. This highlights the need for a centralized platform focused on local connections. Without that platform, residents may feel disconnected from the people and places that are physically closest to them.

Even outside of a local sphere, online forums have become more restricted and private. Some online forums like Discord are fully locked behind an app and offer little customization. Other online platforms, like Reddit, are too open and offer few restrictions for privacy.

We hope to create a platform where both in person and fully online communities can thrive. Maps are a tool offered to communities with in person elements, but it is a fully optional feature.

System Capabilities

Our system should be capable of:

- Allowing users to make forums following community guidelines.
- The ability to make profiles, posts, pins, etc. public/private
- Allow the user to customize their UI (EX: night mode)
- An admin managed forum for moderation. Such as reporting bugs
- Allow users to post within forums, like and comment on other users posts.

- Allow users to tag other users
- Allow users to search for specific forums, users, and tags.
- Our site will host a map.
- Allowing users to drop public/private pins, leave reviews for businesses within their community, see statuses for closures (businesses, roads, etc.) on the map.
- The map will have a legend to allow for easy navigation.
- Our system will host a news section, allowing certain users to post local news articles, comment on news articles, and search for news articles.
- Our system will be capable of moderation such as removing pins, posts, forums, and users from the system.
- Our system will be capable of reporting bugs and users.
- Our system will have documentation to assist users with using the tools we offer.

Business Benefits

It is anticipated that the deployment of this system will provide the following benefits:

- Increase involvement within the community - more people participating in local events and shopping local businesses.
- Increase awareness within the community - safety hazards, road conditions, business closures, etc.
- Maintain current and correct information within the community - Business reviews, business hours of operation, local news stories.

Appendix B: Work Breakdown Structure

The work was divided such that each group member will be responsible for a key subsystem.

Any work outside of the main systems was also evenly distributed.

Chase Brown: The map subsystem

Sean Kehoe: The forum interfacing subsystem

Jennifer Orlando: The moderation subsystem

Mohammed Qudaih: The news subsystem

Rory Strachan: The administrator tools subsystem

References

Discord. (2015). *Discord — A New Way to Chat with Friends & Communities*. Discord.

<https://discord.com/>

Meta Platforms, inc. (2022). Meta | Social Metaverse Company.

https://www.meta.com/about/?utm_source=about.facebook.com&utm_medium=redirect

Reddit. (2005). *Reddit*. Reddit. <https://www.reddit.com/>

Satzinger, J. W., Jackson, R. B., & Burd, S. D. (2016). *Systems Analysis and Design in a Changing World* (7th ed.). Cengage Learning.